



Universidad Politécnica
de Madrid

**Escuela Técnica Superior de
Ingenieros Informáticos**



Máster Universitario en Inteligencia Artificial

Trabajo Fin de Máster

**Entorno de aprendizaje por refuerzo para
el control de UAVs en Aerostack2**

Autor: Jordi Florit Ensenyat
Tutor: Javier Melero

Madrid, Junio - 2026

Este Trabajo Fin de Máster se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su defensa.

Trabajo Fin de Máster

Máster: Máster Universitario en Inteligencia Artificial

Título: Entorno de aprendizaje por refuerzo para el control de UAVs en Aeros-tack2

Junio - 2026

Autor: Jordi Florit Ensenyat

Tutor: Javier Melero

Departamento de Inteligencia Artificial

Escuela Técnica Superior de Ingenieros Informáticos

Universidad Politécnica de Madrid

Índice general

1. Introducción	3
1.1. Motivación del proyecto	3
1.2. Contexto del proyecto	4
1.3. Objetivos	5
1.4. Estructura del documento	5
2. Fundamentos técnicos y trabajo relacionado	7
2.1. Infraestructura robótica: ROS 2 y Aerostack2	7
2.1.1. ROS 2 como middleware robótico	7
2.1.2. Aerostack2	8
2.1.3. Gymnasium y Stable-Baselines3 en el flujo de aprendizaje	10
2.2. Aprendizaje por refuerzo para el control de UAV	10
2.2.1. PPO como algoritmo de entrenamiento	14
2.3. Trabajo relacionado: Simuladores y entornos de entrenamiento para RL en drones	16
2.3.0.0.1. Herramientas de observación y monitorización. . .	17
2.4. Síntesis y posicionamiento del trabajo	18
3. Diseño del sistema	19
3.1. Arquitectura general	19
3.2. Formulación del problema de control	20
3.3. Diseño del entorno Gymnasium	22
3.4. Diseño del reinicio de episodios y accionabilidad	24
3.5. Diseño de la recompensa y condiciones de finalización	25
3.6. Diseño de la ejecución vectorizada	26
4. Implementación	29
4.1. Organización del software	29
4.2. Integración con Aerostack2 y ROS 2	30
4.3. Configuración operativa de PPO	32
4.4. Herramientas de visualización y monitorización	33
4.5. Implementación del entorno de aprendizaje	35
4.6. Diagnóstico del reinicio y de la accionabilidad	36
4.7. Infraestructura de entrenamiento y vectorización	37
5. Pruebas y evaluación	38
5.1. Estrategia de validación	38
5.2. Entorno experimental	39

5.3. Pruebas deterministas y de configuración	39
5.4. Pruebas con Aerostack2 y reinicio de episodios	40
5.5. Métricas y criterios de aceptación	42
6. Resultados	43
6.1. Validación funcional de la infraestructura	43
6.2. Matriz de experimentos PPO	43
6.3. Resultados de entrenamiento y monitorización	44
6.4. Análisis de fallos de integración	45
6.5. Discusión de limitaciones	46
7. Impacto del trabajo	47
7.1. Impacto técnico y académico	47
7.2. Impacto social e industrial	47
7.3. Consideraciones éticas y de seguridad	47
7.4. Relación con los Objetivos de Desarrollo Sostenible	47
8. Conclusiones y líneas futuras	48
8.1. Conclusiones	48
8.2. Líneas futuras	49
Bibliografía	50
A. Anexos	53
A.1. Configuraciones representativas	53
A.2. Comandos representativos	54
A.3. Capturas de TensorBoard	54
A.4. Declaración de uso de herramientas de inteligencia artificial	57

Índice de Figuras

2.1. Comunicación entre nodos en ROS mediante <i>topics</i> y servicios. Fuente: Chollet [Chollet, 2021].	8
2.2. Organización por capas de los componentes software de Aerostack2. Fuente: Fernández-Cortizas et al. [Fernández-Cortizas et al., 2023].	9
2.3. Relación conceptual entre aprendizaje por refuerzo clásico, aprendizaje profundo supervisado y aprendizaje por refuerzo profundo. Fuente: Objectways [Objectways, 2025].	12
2.4. Bucle de interacción entre agente y entorno en aprendizaje por refuerzo. Fuente: AltexSoft [AltexSoft, 2021], basado en la formulación clásica de Sutton y Barto [Sutton y Barto, 2018].	12
2.5. Arquitectura actor-crítico utilizada habitualmente en PPO. Fuente: ResearchGate [ResearchGate, 2024].	15
3.1. Arquitectura general del sistema, separando la lógica de aprendizaje, la interfaz Gymnasium, la comunicación robótica con Aerostack2 y el simulador con servicio de reinicio.	20
3.2. Representación conceptual de la pose de un dron mediante los ángulos de <i>roll</i> , <i>pitch</i> y <i>yaw</i> . Fuente: ResearchGate.	21
3.3. Relación geométrica entre el UAV, el objetivo relativo, la dirección de avance y la orientación de guiñada utilizada en el criterio de <i>path-facing</i>	22
3.4. Ciclo de interacción entre el agente, el entorno Gymnasium y el simulador durante un episodio.	23
3.5. Diseño de la ejecución vectorizada mediante varias instancias lógicas asociadas a espacios de nombres diferenciados sobre una infraestructura de simulación compartida.	28
4.1. Integración implementada entre la política de aprendizaje, la interfaz Gymnasium, Aerostack2 y ROS 2.	31
4.2. Visualización en RViz2 del UAV simulado, la trayectoria reciente y el marcador del objetivo durante una ejecución con Aerostack2.	34
4.3. Ciclo implementado de un episodio de aprendizaje por refuerzo sobre AS2TestEnv.	36
A.1. Métricas de <i>rollout</i> de TensorBoard para Exp010. La tasa de éxito desciende hasta cero y la recompensa media no muestra una tendencia sostenida de mejora, por lo que la figura debe interpretarse como evidencia de diagnóstico y no como demostración de aprendizaje estable.	55

A.2. Métricas de <i>rollout</i> de TensorBoard para Exp011. Aunque la configuración reduce la saturación inicial y mejora parcialmente la recompensa media frente a Exp010, la tasa de éxito vuelve a cero y las terminaciones por límites o baja altitud siguen dominando el entrenamiento.	55
A.3. Métricas internas de PPO para Exp010. Los indicadores de entrenamiento se mantienen en rangos razonables para una ejecución estable de PPO, pero esto no implica éxito de la tarea cuando las métricas de episodio muestran predominio de terminaciones no deseadas.	56
A.4. Métricas internas de PPO para Exp011. La evolución de pérdidas, entropía y varianza explicada se utiliza como apoyo técnico, mientras que la interpretación principal del experimento se obtiene de las métricas de <i>rollout</i> y de los monitores de episodio.	56

Índice de Tablas

2.1. Relación entre las capas tecnológicas empleadas en el presente trabajo.	10
2.2. Diferencias principales entre aprendizaje profundo supervisado y aprendizaje por refuerzo.	11
3.1. Resumen del contrato principal del entorno Gymnasium.	24
3.2. Estados y fallos considerados en el diseño del reinicio de episodios.	25
3.3. Componentes de la función de recompensa y condiciones de finalización del entorno.	27
4.1. Organización principal del software implementado.	30
4.2. Principales parámetros PPO definidos en la configuración base de entrenamiento.	33
4.3. Herramientas principales utilizadas durante la implementación y validación.	35
5.1. Niveles de validación aplicados al entorno desarrollado.	39
5.2. Validadores representativos utilizados durante el desarrollo.	40
5.3. Fallos de integración identificados y mecanismos de control incorporados.	41
6.1. Resumen de experimentos principales y su interpretación.	44
A.1. Configuraciones representativas de la fase de estabilización.	53
A.2. Resumen del uso de herramientas de inteligencia artificial generativa.	57

Índice de Listings

A.1. Ejemplo de ejecución de entrenamiento PPO acotado.	54
A.2. Ejemplo de validación de accionabilidad con acción fija.	54
A.3. Ejemplo de entrenamiento supervisado con reanudación.	54

Resumen

«Aquí va el resumen del Trabajo Fin de Máster. Extensión máxima 2 páginas.»

Palabras Clave:

Abstract

«Abstract of the Master's Thesis. Maximum length: 2 pages.»

Keywords:

Capítulo 1

Introducción

Este capítulo presenta el marco general del Trabajo Fin de Máster, centrado en la aplicación de aprendizaje por refuerzo al control de vehículos aéreos no tripulados dentro del ecosistema Aerostack2. En primer lugar se expone la motivación del trabajo y su relevancia en el contexto actual de la robótica aérea autónoma. A continuación se describe el contexto técnico y de investigación en el que se enmarca el desarrollo realizado. Posteriormente se definen el objetivo general y los objetivos específicos. Por último, se resume la organización de la memoria.

1.1. Motivación del proyecto

Los vehículos aéreos no tripulados han pasado en pocos años de ser una tecnología principalmente experimental a convertirse en una herramienta útil en aplicaciones industriales, logísticas, de inspección y competición. En estos escenarios, la autonomía y la capacidad de respuesta del vehículo son aspectos críticos, especialmente cuando el dron debe seguir referencias espaciales, reaccionar ante cambios del entorno o mantener un comportamiento estable en maniobras rápidas.

Dentro de este contexto, el control clásico basado en reguladores PID sigue siendo una solución ampliamente utilizada por su simplicidad, interpretabilidad y robustez. Sin embargo, cuando la dinámica del sistema es no lineal, el régimen de operación es exigente o se desea adaptar el comportamiento a criterios de tarea más amplios, el ajuste manual de estos controladores puede resultar costoso y difícil de generalizar. Frente a ello, el aprendizaje por refuerzo ofrece una alternativa de interés: permite que una política de control aprenda a actuar a partir de la interacción con un entorno, optimizando un criterio de comportamiento definido mediante recompensas.

La motivación de este trabajo surge en la intersección entre robótica aérea e inteligencia artificial. En el grupo CVAR (*Computer Vision and Aerial Robotics*) existe interés en desarrollar soluciones de control cada vez más ágiles para drones, en particular en el marco de la competición A2RL, donde el rendimiento del vehículo depende de su capacidad para ejecutar maniobras rápidas y precisas. Para que una aproximación basada en aprendizaje por refuerzo sea viable, no basta con seleccionar un algoritmo de entrenamiento sino que resulta imprescindible disponer de una infraestructura experimental que conecte de forma correcta el simulador, el *middleware* robótico y las herramientas de aprendizaje automático.

Este Trabajo Fin de Máster responde a esa necesidad mediante la construcción de un entorno reproducible que integra Gymnasium, Stable-Baselines3, ROS 2 y Aerostack2. La aportación principal no se formula todavía como una demostración de superioridad del aprendizaje por refuerzo frente a un controlador clásico, sino como la base técnica necesaria para entrenar, diagnosticar y evaluar políticas de control en condiciones controladas y extensibles. En consecuencia, el alcance del trabajo prioriza la reproducibilidad de la infraestructura, la validez del ciclo de episodio y la disponibilidad de diagnósticos verificables frente a afirmaciones de aprendizaje robusto que no estén respaldadas por experimentos completos.

1.2. Contexto del proyecto

El trabajo se desarrolla sobre Aerostack2, un *framework* para robótica aérea basado en ROS 2 que proporciona componentes reutilizables para control, estimación, comportamientos de vuelo e integración con diferentes plataformas y simuladores. En particular, se utiliza el simulador dinámico de multirrotores de Aerostack2, una herramienta sin interfaz gráfica que modela la dinámica del vehículo y que puede visualizarse mediante RViz2. Al no requerir interfaz gráfica, resulta adecuado para plantear experimentos con varios drones dentro de un mismo simulador compartido, aunque la validación disponible hasta el momento debe entenderse como validación de infraestructura y no como evidencia de aceleración de entrenamiento ni de rendimiento final.

El repositorio `rl_uav_aerostack2` contiene el entorno `AS2TestEnv-v0`, registrado en `rl_uav/__init__.py` e implementado en `rl_uav/envs/as2_test_env.py`. Este entorno encapsula el ciclo básico de Gymnasium, inicializa la interfaz `DroneInterface` de Aerostack2, arma el dron, activa el modo *offboard*, ejecuta el despegue y envía comandos de velocidad mediante Aerostack2 y ROS 2. La acción del agente es un vector $[v_x, v_y, v_z, v_{yaw}]$, compuesto por velocidades lineales y una velocidad angular en yaw.

La observación principal del entorno no está formada por posición y velocidad absolutas, sino por el vector normalizado $[dx, dy, dz, d_{yaw}]$, que representa la diferencia de posición y orientación respecto al objetivo. La velocidad del dron se utiliza cuando procede como dato auxiliar para recompensa, diagnóstico y cálculo del término de *path-facing*, pero no forma parte de la observación principal del agente. La recompensa combina una penalización continua por distancia al objetivo con un término de alineación entre el yaw del dron y la dirección horizontal de movimiento. La finalización del episodio contempla tres casos: éxito al alcanzar una distancia inferior al umbral definido, salida de los límites del escenario y truncado por superar el número máximo de pasos.

Sobre esta base se ha preparado infraestructura de entrenamiento con PPO mediante Stable-Baselines3, incluyendo el archivo de configuración `configs/train_ppo.yaml`, scripts de entrenamiento, registro de métricas, *checkpoints* y soporte para entornos vectorizados. (También existe una configuración de referencia basada en PID que puede servir como línea base en la evaluación posterior. No obstante, el estado actual del trabajo exige distinguir con precisión entre tres niveles: la infraestructura implementada y validada, el entrenamiento PPO preparado para su ejecución sistemática y la comparación experimental con PID, que queda planteada como evaluación prevista

y no como un resultado ya demostrado.) <- Idea inicial del tfm, a día de hoy no se si sera posible hacer la comparativa con PID, por lo que lo dejo como previsto y no como hecho.

1.3. Objetivos

El objetivo general de este Trabajo Fin de Máster es desarrollar una infraestructura reproducible para formular, entrenar y evaluar controladores de alto nivel para drones mediante aprendizaje por refuerzo en un entorno de simulación integrado con un *stack* robótico.

Para alcanzar este objetivo general, el trabajo se organiza en los siguientes objetivos específicos:

- Analizar el problema de control de alto nivel de un dron en simulación, identificando las variables de observación, las acciones de velocidad y las condiciones de terminación necesarias para formularlo como un entorno de aprendizaje por refuerzo.
- Diseñar e implementar un entorno de aprendizaje que integre el simulador dinámico con el *middleware* robótico y permita ejecutar el ciclo de interacción entre agente y dron de forma reproducible.
- Incorporar una función de recompensa coherente con el seguimiento de objetivos espaciales y la restricción de *path-facing*, manteniendo información de diagnóstico suficiente para analizar el comportamiento del entorno.
- Preparar la infraestructura de entrenamiento con PPO y soporte para vectorización sincrónica de múltiples drones dentro de un simulador compartido, diferenciando la validación funcional de la infraestructura de la evaluación posterior del aprendizaje.
- Analizar las limitaciones de integración observadas durante los experimentos, especialmente las asociadas al reinicio de episodios, la accionabilidad de los comandos y la reproducibilidad física del simulador.
- Establecer una base experimental para evaluar el controlador aprendido y compararlo con una configuración PID de referencia cuando se disponga de ejecuciones completas y métricas representativas. <- (Idea inicial del tfm, a día de hoy no se si sera posible hacer la comparativa con PID, por lo que lo dejo como previsto y no como hecho.)

1.4. Estructura del documento

La memoria se organiza siguiendo una progresión desde el contexto técnico hasta la evaluación del sistema construido. Tras esta introducción, el Capítulo 2 presenta el estado del arte y el trabajo relacionado sobre marcos software para robótica aérea, aprendizaje por refuerzo aplicado a UAV y simuladores de entrenamiento. El Capítulo 3 desarrolla el diseño de la solución, incluyendo la formulación del entorno, las decisiones de observación, acción, recompensa y terminación, y la estrategia de integración con Aerostack2.

El Capítulo 4 describe la implementación realizada, con atención a la estructura del repositorio, la conexión con ROS 2, la infraestructura PPO y el soporte de vectorización. El Capítulo 5 recoge las pruebas de validación funcional de la infraestructura y los criterios usados para comprobar que el entorno se comporta de forma consistente. El Capítulo 6 presenta los resultados disponibles y su discusión, distinguiendo entre validaciones de infraestructura y resultados de aprendizaje que requieran experimentación posterior. El Capítulo 7 analiza el impacto del trabajo desde una perspectiva técnica y académica. Finalmente, el Capítulo 8 expone las conclusiones, revisa el grado de cumplimiento de los objetivos y plantea líneas futuras de desarrollo.

Capítulo 2

Fundamentos técnicos y trabajo relacionado

Este capítulo introduce los fundamentos técnicos necesarios para comprender el entorno desarrollado y revisa los trabajos relacionados con aprendizaje por refuerzo aplicado a vehículos aéreos no tripulados. No se pretende realizar una revisión exhaustiva de todos los conceptos de robótica o aprendizaje automático, sino seleccionar aquellos elementos que condicionan directamente el diseño posterior como el middleware robótico empleado, la arquitectura aérea sobre la que se ejecuta el sistema, el papel de la simulación y la formulación del problema de aprendizaje.

La organización sigue una progresión de infraestructura a método. En primer lugar, se presentan ROS 2 y Aerostack2 como base software para construir sistemas aéreos autónomos. A continuación, se revisa el aprendizaje por refuerzo profundo en drones y se justifica el uso de PPO como algoritmo de entrenamiento. Después se analizan simuladores y entornos de entrenamiento relevantes, con especial atención a las implicaciones de usar un simulador integrado en un *stack* robótico. Finalmente, se sintetiza el posicionamiento del presente trabajo respecto a la literatura revisada.

2.1. Infraestructura robótica: ROS 2 y Aerostack2

El desarrollo de sistemas aéreos autónomos no consiste únicamente en programar un algoritmo de control. Un UAV operativo requiere coordinar estimación de estado, control, comunicaciones, gestión de modos de vuelo, seguridad, planificación y, en muchos casos, percepción. Esta integración es uno de los principales motivos por los que los trabajos de robótica suelen apoyarse en arquitecturas software reutilizables que permiten que los algoritmos se conecten con un sistema completo en lugar de ejecutarse como componentes aislados.

2.1.1. ROS 2 como middleware robótico

ROS 2 proporciona una infraestructura de comunicación para sistemas robóticos distribuidos. Un sistema ROS 2 se organiza como un grafo de nodos que intercambian información mediante distintos mecanismos de comunicación. Los *topics* permiten publicar flujos continuos de datos, los servicios modelan interacciones de petición y respuesta, y las acciones representan tareas de mayor duración con realimentación

y posibilidad de cancelación [Open Robotics, 2026]. Esta separación resulta especialmente útil en robótica aérea, donde conviven señales continuas de estado con órdenes discretas de ciclo de vida, como armar, despegar o aterrizar.

La Figura 2.1 muestra esta idea de forma conceptual. Un nodo puede actuar como publicador o suscriptor de un *topic*, lo que permite distribuir información de estado o medidas de sensores. El mismo nodo también puede ofrecer o consumir servicios, adecuados para operaciones puntuales que requieren una respuesta explícita. Esta distinción es importante en el presente trabajo porque el ciclo de aprendizaje combina ambos tipos de interacción: lectura recurrente del estado del UAV y ejecución de órdenes de preparación o control sobre la plataforma simulada.

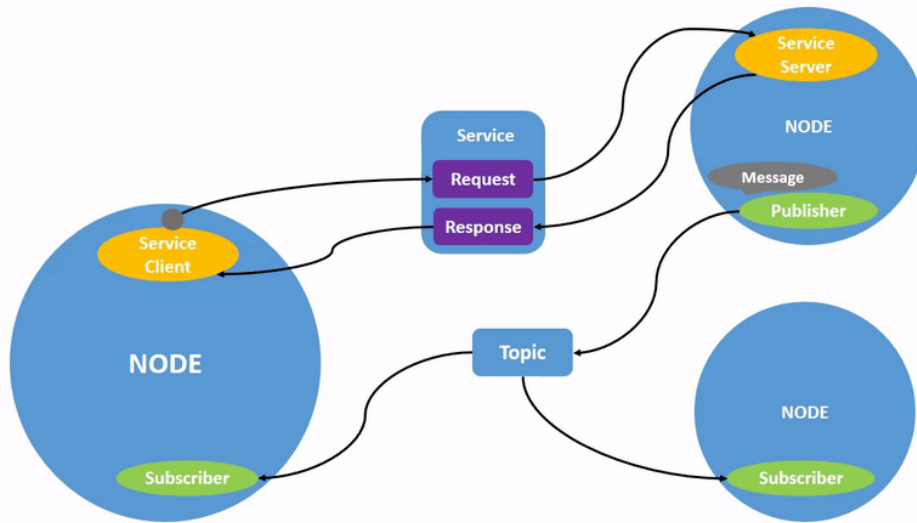


Figura 2.1: Comunicación entre nodos en ROS mediante *topics* y servicios. Fuente: Chollet [Chollet, 2021].

Desde el punto de vista de este trabajo, ROS 2 se usa como la capa que permite comunicar el entorno de aprendizaje con el sistema robótico simulado. La política de aprendizaje por refuerzo no publica directamente en todos los *topics* ni gestiona todos los servicios disponibles; esa responsabilidad queda encapsulada en las capas de Aerostack2 y en la interfaz desarrollada. Esta decisión evita que el algoritmo de aprendizaje dependa de detalles de comunicación de bajo nivel.

2.1.2. Aerostack2

Sobre esta base se sitúa Aerostack2. Inicialmente Aerostack fue propuesto como un marco software abierto, modular y multicapa para el desarrollo de sistemas aéreos autónomos complejos [Sánchez-López et al., 2017]. Su objetivo era proporcionar una arquitectura reutilizable capaz de integrar componentes de control, percepción, planificación y ejecución de misiones en robótica aérea. Esta idea es relevante porque desplaza el foco desde el algoritmo aislado hacia el sistema completo que permite ejecutar una misión aérea.

Por otra parte, Aerostack2 constituye la evolución de esta línea sobre ROS 2. Su

Fundamentos técnicos y trabajo relacionado

diseño enfatiza la independencia de plataforma, la arquitectura basada en *plugins*, el soporte para sistemas multirrobot y la definición de misiones mediante comportamientos reutilizables [Fernández-Cortizas et al., 2023]. En lugar de imponer un único simulador o una única plataforma física, Aerostack2 actúa como capa de abstracción para conectar distintos vehículos, simuladores y componentes de misión. Esta característica lo hace adecuado para trabajos en los que interesa aproximar la experimentación en simulación a un flujo de robótica aérea operativo.

La arquitectura de Aerostack2 puede entenderse como una organización por capas, resumida en la Figura 2.2. En los niveles inferiores se sitúan el middleware, los mensajes de comunicación y las interfaces con plataformas o sensores. Sobre ellos se construyen funciones robóticas básicas, como estimación de estado, control de movimiento o gestión de emergencias. A un nivel superior aparecen los comportamientos de robot, como despegue, aterrizaje, seguimiento de trayectorias o detección de marcadores. Finalmente, las capas de misión y aplicación permiten componer esas capacidades para casos de uso concretos.

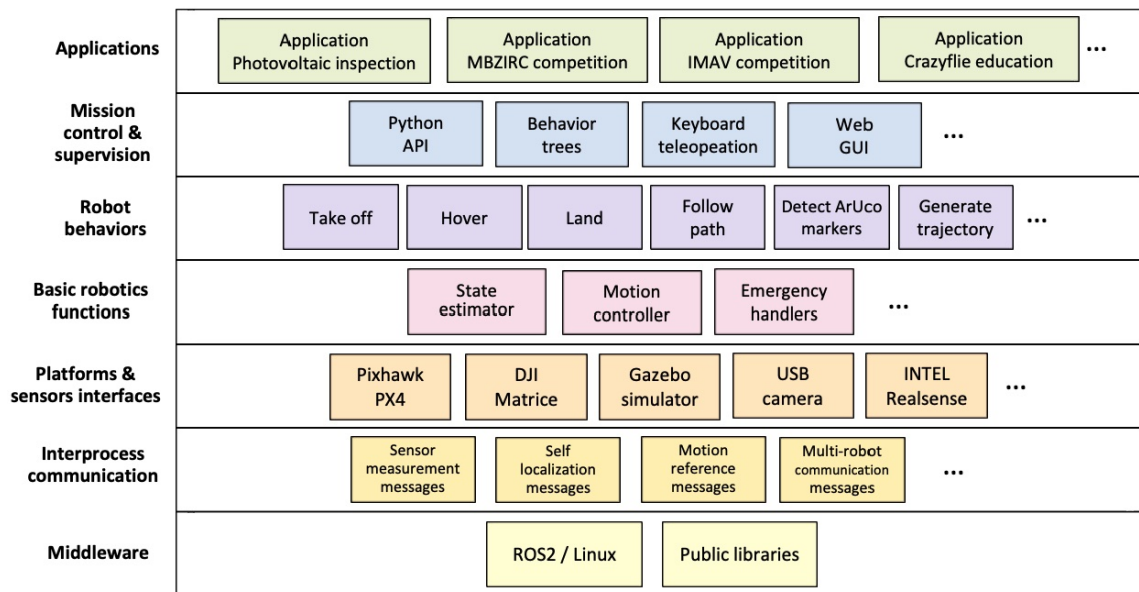


Figura 2.2: Organización por capas de los componentes software de Aerostack2. Fuente: Fernández-Cortizas et al. [Fernández-Cortizas et al., 2023].

Esta organización por capas ayuda a explicar por qué Aerostack2 resulta especialmente útil para el presente trabajo. El entorno de aprendizaje por refuerzo no sustituye a los controladores, estimadores o comportamientos de Aerostack2, sino que se conecta por encima de ellos para emitir consignas de velocidad dentro de una infraestructura robótica ya existente. Así, el problema de aprendizaje se mantiene centrado en decidir acciones de alto nivel, mientras que la ejecución robótica queda delegada en componentes especializados del *stack*.

2.2. Aprendizaje por refuerzo para el control de UAV

Nivel	Función principal	Papel en este trabajo
ROS 2	Comunicación entre nodos mediante <i>topics</i> , servicios y acciones.	Transporte de órdenes, estados y operaciones de ciclo de vida.
Aerostack2	Arquitectura aérea sobre ROS 2 con interfaces y comportamientos reutilizables.	Sustrato robótico usado por el entorno de aprendizaje.
Simulador AS2	Ejecución simulada del multirrotor y de sus interfaces robóticas.	Fuente de experiencia para entrenamiento y validación funcional.
Gymnasium/SB3	Interfaz estándar de aprendizaje por refuerzo y entrenamiento PPO.	Capa de interacción del agente con observaciones, acciones y recompensas.

Cuadro 2.1: Relación entre las capas tecnológicas empleadas en el presente trabajo.

El Cuadro 2.1 resume la función de cada capa. Esta separación ayuda a delimitar la contribución del trabajo. No se propone un nuevo middleware robótico ni una nueva arquitectura aérea, sino una interfaz reproducible entre un entorno de aprendizaje por refuerzo y una plataforma aérea basada en ROS 2 y Aerostack2. Por ello, los capítulos posteriores distinguen entre la infraestructura robótica reutilizada, el diseño del problema de aprendizaje y la implementación del entorno Gymnasium.

2.1.3. Gymnasium y Stable-Baselines3 en el flujo de aprendizaje

Gymnasium proporciona una API común para definir entornos de aprendizaje por refuerzo mediante operaciones como `reset` y `step`, junto con espacios de observación y acción explícitos [Towers et al., 2024; Farama Foundation, 2026]. Su papel en este trabajo no es simular por sí mismo la dinámica del UAV, sino ofrecer el contrato mediante el cual un algoritmo de aprendizaje puede interactuar de forma uniforme con una tarea implementada sobre ROS 2 y Aerostack2. Esta separación permite que la lógica del agente no dependa directamente de los detalles de comunicación robótica.

Stable-Baselines3 aporta implementaciones mantenidas de algoritmos de aprendizaje por refuerzo profundo y utilidades de entrenamiento compatibles con este tipo de entornos, incluida la implementación de PPO utilizada más adelante [Stable-Baselines3 Contributors, 2026]. En el presente trabajo se emplea como capa de entrenamiento y monitorización, mientras que la definición del problema, la recompensa, los reinicios y las condiciones de seguridad permanecen en el entorno desarrollado. Por tanto, Gymnasium estandariza la interfaz y Stable-Baselines3 proporciona el algoritmo, pero la validez experimental depende de que la adaptación robótica produzca transiciones físicas fiables.

2.2. Aprendizaje por refuerzo para el control de UAV

El aprendizaje por refuerzo se diferencia de otros enfoques de aprendizaje automático porque el conocimiento no se obtiene a partir de un conjunto fijo de pares entrada-salida, sino mediante interacción secuencial con un entorno. En aprendizaje supervisado clásico, una red neuronal aprende a aproximar una función usando ejemplos

etiquetados; en aprendizaje por refuerzo, en cambio, el agente debe actuar, observar las consecuencias de sus acciones y ajustar su comportamiento según una señal de recompensa. Esta diferencia es esencial en robótica: un robot no solo debe reconocer patrones, sino decidir acciones que modifican el estado físico del sistema.

Esta diferencia puede resumirse en el Cuadro 2.2. En aprendizaje profundo supervisado, el objetivo habitual es aprender una relación estática entre entradas y etiquetas. En aprendizaje por refuerzo, el problema es dinámico las decisiones modifican los datos que se observarán después. Por ello, la calidad del aprendizaje no depende solo del modelo neuronal, sino también de la definición del entorno, de la recompensa y de la estrategia de exploración [Sutton y Barto, 2018; Molina, 2025b].

Aspecto	Aprendizaje profundo supervisado	Aprendizaje por refuerzo
Datos de entrenamiento	Conjunto de ejemplos normalmente fijo y etiquetado.	Experiencia generada por interacción con el entorno.
Señal de aprendizaje	Error entre predicción y etiqueta conocida.	Recompensa escalar y acumulativa.
Salida del modelo	Predicción, clasificación o regresión.	Acción o distribución de acciones.
Dependencia temporal	Las muestras suelen tratarse como ejemplos independientes.	Cada acción condiciona los estados y recompensas futuros.
Relevancia en robótica	Percepción, estimación o reconocimiento.	Control, navegación y toma de decisiones secuencial.

Cuadro 2.2: Diferencias principales entre aprendizaje profundo supervisado y aprendizaje por refuerzo.

La Figura 2.3 complementa esta comparación. El aprendizaje por refuerzo clásico puede representarse mediante tablas de valor cuando los estados y acciones son discretos y de tamaño limitado. El aprendizaje profundo supervisado, por su parte, utiliza redes neuronales para extraer patrones a partir de datos etiquetados. El aprendizaje por refuerzo profundo combina ambas ideas: emplea redes neuronales para aproximar políticas, funciones de valor o ambas, pero mantiene el ciclo de interacción con el entorno propio del aprendizaje por refuerzo. Esta combinación es la que permite aplicar RL a problemas con estados continuos, observaciones de alta dimensión o espacios de acción complejos, como los que aparecen en robótica aérea.

2.2. Aprendizaje por refuerzo para el control de UAV

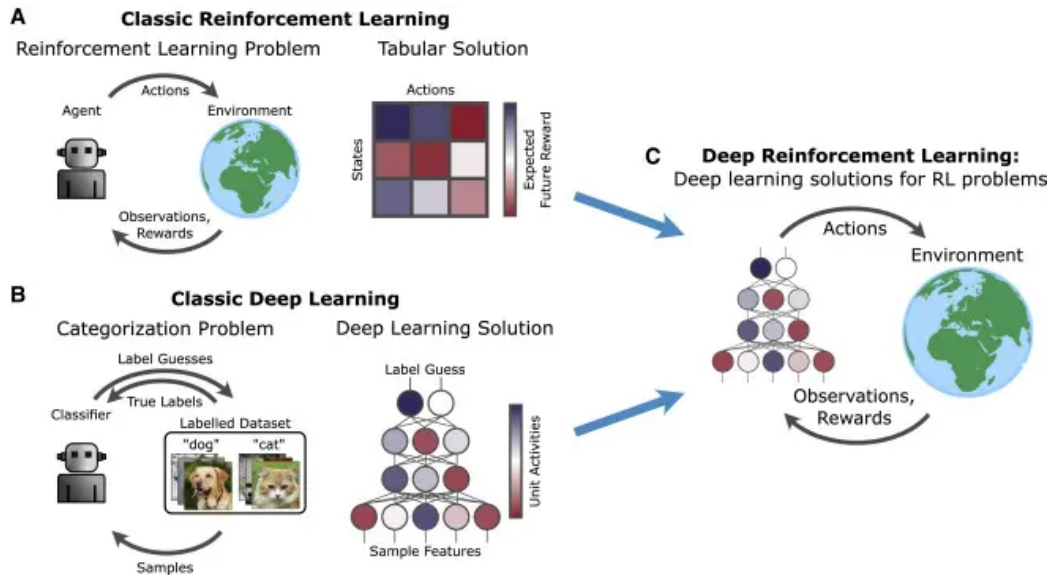


Figura 2.3: Relación conceptual entre aprendizaje por refuerzo clásico, aprendizaje profundo supervisado y aprendizaje por refuerzo profundo. Fuente: Objectways [Objectways, 2025].

La Figura 2.4 resume el ciclo básico de aprendizaje por refuerzo. En cada instante t , el agente recibe un estado u observación S_t , selecciona una acción A_t y el entorno responde con una nueva situación S_{t+1} y una recompensa R_{t+1} [Sutton y Barto, 2018]. En un problema robótico, el entorno incluye tanto la dinámica del vehículo como la infraestructura que ejecuta las órdenes y proporciona medidas del estado. En este trabajo, ese papel lo desempeñan conjuntamente Aerostack2, ROS 2 y el simulador multirrotor.

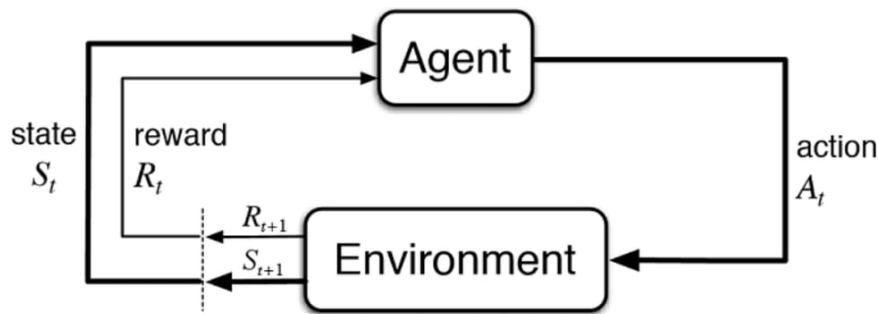


Figura 2.4: Bucle de interacción entre agente y entorno en aprendizaje por refuerzo. Fuente: AltexSoft [AltexSoft, 2021], basado en la formulación clásica de Sutton y Barto [Sutton y Barto, 2018].

Formalmente, un problema de aprendizaje por refuerzo suele expresarse como un proceso de decisión de Markov. Esta formulación se define mediante un conjunto de estados S , un conjunto de acciones A , una función de transición P , una función de recompensa R y un factor de descuento $\gamma \in [0, 1]$ [Sutton y Barto, 2018]. La política π determina cómo actúa el agente: puede representarse como una función determinista

$\pi(s) = a$ o como una distribución de probabilidad $\pi(a|s)$ sobre acciones posibles. El objetivo del aprendizaje consiste en encontrar una política que maximice la recompensa acumulada esperada.

El retorno descontado desde un instante t se define como:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}.$$

Este valor resume la utilidad futura de una secuencia de decisiones. El factor de descuento γ permite ponderar las recompensas lejanas: valores cercanos a uno consideran horizontes más largos, mientras que valores menores favorecen recompensas inmediatas. En robótica aérea esta distinción es importante, porque una acción puede parecer buena a corto plazo, por ejemplo aumentar la velocidad hacia el objetivo, pero generar trayectorias inestables o salidas de la región segura en pasos posteriores.

A partir del retorno se definen dos funciones habituales. La función de valor $V^\pi(s)$ estima el retorno esperado al seguir la política π desde un estado s . La función de valor-acción $Q^\pi(s, a)$ estima el retorno esperado al ejecutar una acción concreta a en el estado s y continuar después con la política π . La diferencia entre ambas magnitudes permite definir la ventaja:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

La ventaja mide si una acción concreta es mejor o peor que el comportamiento medio esperado en ese estado. Esta idea será relevante para PPO, ya que el algoritmo usa estimaciones de ventaja para decidir en qué dirección debe modificarse la política.

En el contexto de vehículos aéreos no tripulados, el aprendizaje por refuerzo resulta atractivo porque permite optimizar comportamientos difíciles de programar mediante reglas cerradas. Las revisiones recientes muestran aplicaciones en planificación, navegación y control de UAV, especialmente en escenarios con dinámica no lineal, espacios de acción continuos o transferencia desde simulación [Azar et al., 2021; Chan et al., 2024]. Sin embargo, estas mismas revisiones subrayan limitaciones importantes: necesidad de muchas interacciones, sensibilidad al diseño de la recompensa, dependencia de la fidelidad del simulador y dificultad de trasladar políticas entrenadas a plataformas reales.

Una parte significativa de los trabajos se ha centrado en control a bajo nivel, es decir, en aprender políticas que actúan cerca de los actuadores o de las consignas de actitud. Pi et al. propusieron un controlador de bajo nivel basado en aprendizaje por refuerzo capaz de generalizar a vehículos multirrotor con distintas configuraciones físicas [Pi et al., 2021]. En una línea relacionada, Yoo et al. combinaron controladores deterministas clásicos con una política aprendida, mostrando que una aproximación híbrida puede mejorar la convergencia y el seguimiento de trayectorias mediante control en velocidad [Yoo et al., 2021]. Estos trabajos son relevantes para una posible extensión jerárquica del presente trabajo, aunque el entorno desarrollado se sitúa actualmente en un nivel de control más alto: la política genera comandos de velocidad lineal y velocidad angular en yaw, no señales de motor ni tasas angulares completas.

Otra línea activa es la del vuelo agresivo y el empuje del dron hacia sus límites dinámicos. Kaufmann et al. demostraron en *Deep Drone Acrobatics* que es posible entrenar

en simulación una política para realizar maniobras acrobáticas y transferirla al dron real sin ajuste fino posterior [Kaufmann et al., 2020]. Más adelante, los mismos autores analizaron comparativamente distintas abstracciones de control y observaron que las políticas que actúan sobre *body-rates* y *thrust* resultan más robustas en transferencia que aquellas que operan directamente sobre los motores [Kaufmann et al., 2022]. Este punto muestra que el nivel de abstracción de la acción no es una decisión secundaria, sino un elemento central del diseño del problema de aprendizaje.

Los resultados recientes en vuelo agresivo refuerzan esta idea. Eschmann et al. mostraron que, con una arquitectura adecuada y un simulador muy optimizado, es posible entrenar controladores de bajo nivel en tiempos extraordinariamente reducidos y mantener capacidad de transferencia a plataformas reales [Eschmann et al., 2024]. En paralelo, Song et al. estudiaron de manera sistemática la competición entre control óptimo y aprendizaje por refuerzo en un problema de carrera autónoma con drones, concluyendo que el aprendizaje por refuerzo puede superar al control óptimo cuando el objetivo se formula directamente a nivel de tarea y se entrena con suficiente robustez frente a incertidumbres del modelo [Song et al., 2023]. Este resultado es pertinente para el contexto de A2RL que motiva el trabajo, aunque debe interpretarse como antecedente de la literatura y no como evidencia del rendimiento del entorno aquí desarrollado.

La literatura revisada revela, por tanto, una diversidad de formulaciones. Muchos trabajos se centran en estabilización de actitud, control a nivel de actuadores, vuelo acrobático o navegación *end-to-end* basada en percepción [Kaufmann et al., 2020; Pi et al., 2021; Yoo et al., 2021]. A partir de esas obras puede inferirse que existe menos atención a controladores intermedios integrados en un *stack* robótico de ROS 2, donde la acción se expresa como $[v_x, v_y, v_z, v_{yaw}]$ y la observación principal como un estado relativo normalizado $[dx, dy, dz, d_{yaw}]$. Esta inferencia ayuda a delimitar el espacio de contribución del presente trabajo sin convertirlo en una afirmación absoluta sobre toda la literatura existente.

2.2.1. PPO como algoritmo de entrenamiento

PPO, *Proximal Policy Optimization*, pertenece a la familia de métodos de gradiente de política. Estos métodos no aprenden únicamente una función que valora acciones, sino que optimizan directamente una política parametrizada $\pi_\theta(a|s)$ [Schulman et al., 2017]. En aprendizaje por refuerzo profundo, esta política se implementa habitualmente mediante una red neuronal que recibe una observación y produce una distribución de acciones. En espacios continuos, como el empleado en este trabajo, la acción puede interpretarse como una muestra o una salida de una distribución sobre velocidades admisibles.

PPO suele implementarse con una arquitectura actor-crítico. El actor representa la política y decide qué acción ejecutar. El crítico estima el valor de los estados o de las acciones y proporciona una señal auxiliar para evaluar si la acción tomada fue mejor o peor de lo esperado. La Figura 2.5 muestra este esquema: las trayectorias generadas por la interacción con el entorno se usan para actualizar tanto el crítico como la política del actor. Esta separación es útil porque el actor aprende a decidir, mientras que el crítico reduce la varianza de la estimación del gradiente mediante información de valor.

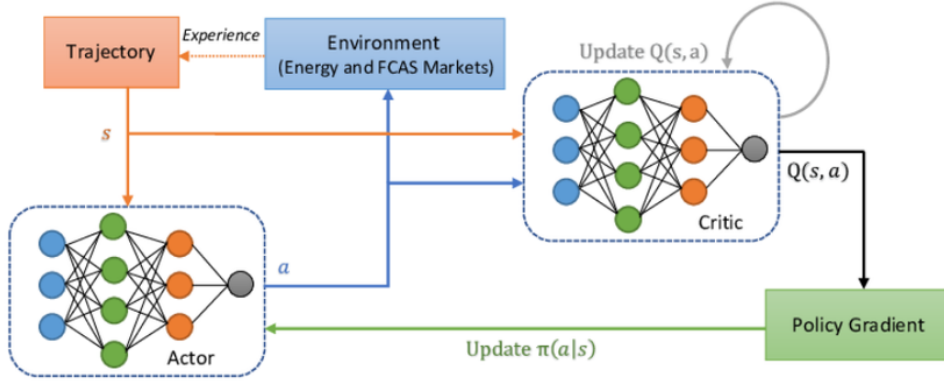


Figura 2.5: Arquitectura actor-crítico utilizada habitualmente en PPO. Fuente: ResearchGate [ResearchGate, 2024].

En una arquitectura actor-crítico, el crítico permite estimar la ventaja sin esperar siempre al retorno completo de un episodio. Una aproximación habitual es partir del error temporal:

$$\delta_t = R_{t+1} + \gamma V(s_{t+1}) - V(s_t),$$

que compara la recompensa inmediata más el valor estimado del estado siguiente con el valor estimado del estado actual. En implementaciones modernas de PPO se emplea frecuentemente una estimación de ventaja generalizada, que combina varios errores temporales ponderados:

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}.$$

El parámetro λ controla el compromiso entre sesgo y varianza: valores altos incorporan más información de retornos largos, mientras que valores bajos se apoyan más en el valor estimado por el crítico. Esta explicación es relevante para interpretar parámetros como `gamma` y `gae_lambda` en la configuración de entrenamiento del presente trabajo [Molina, 2025a; Molina, 2025c].

La dificultad principal de los métodos de gradiente de política es que una actualización demasiado grande puede destruir una política que estaba funcionando razonablemente bien. PPO aborda este problema comparando la política nueva con la política anterior mediante la razón de probabilidades:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}.$$

Si esta razón se aleja demasiado de uno, significa que la nueva política asigna una probabilidad muy distinta a la acción tomada respecto a la política que generó los datos. PPO limita ese cambio mediante una función objetivo recortada:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right].$$

El parámetro ϵ define cuánto puede separarse la nueva política de la anterior en una actualización. Si la ventaja estimada $\hat{A}_t$ es positiva, la actualización aumenta la probabilidad de la acción, pero solo hasta el límite permitido por el recorte. Si la ventaja

2.3. Trabajo relacionado: Simuladores y entornos de entrenamiento para RL en drones

es negativa, reduce esa probabilidad, de nuevo sin permitir cambios excesivos. Este mecanismo proporciona una forma práctica de mantener las actualizaciones cerca de la política que generó las muestras, evitando saltos demasiado bruscos [Schulman et al., 2017].

El funcionamiento práctico de PPO puede resumirse como un ciclo de cinco pasos. Primero, la política actual interactúa con el entorno y genera trayectorias de estados, acciones, recompensas y finales de episodio. Segundo, el crítico estima valores y se calculan ventajas para cada transición. Tercero, se actualiza el actor maximizando la función objetivo recortada. Cuarto, se entrena el crítico para reducir el error entre sus valores estimados y los retornos objetivo. Quinto, se puede añadir un término de entropía para favorecer exploración y evitar que la política se vuelva determinista demasiado pronto. En la práctica, muchas implementaciones optimizan una combinación de pérdida de política, pérdida de valor y bonificación de entropía [Schulman et al., 2017; Stable-Baselines3 Contributors, 2026]:

$$L(\theta) = L^{CLIP}(\theta) - c_1 L^{VF}(\theta) + c_2 S[\pi_\theta].$$

Aquí, L^{VF} representa el error de la función de valor, $S[\pi_\theta]$ la entropía de la política, y c_1 y c_2 ponderan la importancia relativa de ambos términos. Esta expresión permite entender por qué la configuración de PPO incluye parámetros específicos para la función de valor, la exploración y la norma del gradiente, no se entrena solo una red que “elige acciones”, sino un sistema de optimización que equilibra mejora de política, estimación de valor y estabilidad.

En el presente trabajo, PPO resulta adecuado por tres motivos. En primer lugar, el espacio de acciones es continuo y acotado, ya que la política genera velocidades lineales y velocidad de guiñada. En segundo lugar, su implementación en Stable-Baselines3 se integra de forma directa con entornos Gymnasium y entornos vectorizados. En tercer lugar, el mecanismo de *clipping* reduce el riesgo de actualizaciones inestables, un aspecto relevante cuando cada muestra requiere interactuar con un simulador robótico [Stable-Baselines3 Contributors, 2026]. La elección de PPO debe entenderse, por tanto, como una decisión de diseño para construir un pipeline experimental estable, no como evidencia previa de que la política aprendida supere a un controlador clásico.

Desde una perspectiva robótica, PPO también impone una restricción metodológica: la calidad del aprendizaje depende de la definición del entorno. Observaciones mal normalizadas, acciones con escalas inadecuadas, recompensas ambiguas o reinicios poco fiables pueden producir políticas aparentemente entrenadas pero incapaces de resolver la tarea. Por ello, los capítulos posteriores no se limitan a indicar que se usa PPO, sino que describen explícitamente el contrato Gymnasium, la formulación de observaciones y acciones, la recompensa, las condiciones de finalización y la infraestructura de validación.

2.3. Trabajo relacionado: Simuladores y entornos de entrenamiento para RL en drones

El entrenamiento mediante aprendizaje por refuerzo en robótica aérea depende fuertemente del simulador empleado. Un simulador adecuado debe ofrecer fidelidad dinámica suficiente para capturar el comportamiento del vehículo, pero también un coste

computacional reducido que permita generar experiencia de entrenamiento [Chan et al., 2024]. La literatura reciente coincide en que este equilibrio entre realismo, velocidad y facilidad de integración es uno de los factores determinantes del éxito de una solución basada en aprendizaje por refuerzo [Azar et al., 2021; Chan et al., 2024].

Entre los entornos influyentes se encuentra Flightmare, un simulador de cuadricópteros diseñado para desacoplar el renderizado y la física, lo que permite combinar flexibilidad con una velocidad de simulación elevada [Song et al., 2021]. Su interés para aprendizaje por refuerzo reside en que puede simular centenares de drones en paralelo, lo cual lo convierte en una referencia cuando el objetivo es maximizar el rendimiento de muestreo. Desde otra perspectiva, Air Learning amplió el uso de AirSim con un entorno tipo Gym para navegación autónoma, introduciendo aleatorización del dominio, *hardware-in-the-loop* y métricas de “calidad de vuelo” orientadas al despliegue en plataformas con recursos limitados [Krishnan et al., 2021]. Este trabajo muestra que el valor de un simulador para aprendizaje por refuerzo no depende solo de la dinámica, sino también de su capacidad para aproximar las restricciones computacionales del sistema final.

En una dirección más ligera y abierta, Panerati et al. propusieron un entorno Gym basado en PyBullet para cuadricópteros individuales y multiagente, explícitamente orientado a facilitar la comparación entre enfoques de control clásico y de aprendizaje [Panerati et al., 2021]. Su aportación es especialmente interesante para trabajos académicos porque reduce la fricción de entrada, ofrece portabilidad y proporciona ejemplos tanto de control PID como de tareas de aprendizaje por refuerzo. Finalmente, *Learning to Fly in Seconds* plantea un simulador extremadamente optimizado para entrenamiento de controladores de bajo nivel, subrayando que la eficiencia computacional puede llegar a ser tan importante como la fidelidad dinámica cuando el objetivo es acelerar el ciclo de investigación [Eschmann et al., 2024].

Las revisiones de conjunto confirman esta diversidad de soluciones. Chan et al. comparan simuladores orientados a aprendizaje por refuerzo desde la perspectiva de sus prestaciones, interfaces y capacidades de integración, y concluyen que no existe una herramienta universalmente superior, sino distintas elecciones dependiendo del problema, del tipo de aprendizaje y del nivel de realismo requerido [Chan et al., 2024]. Desde la perspectiva de este trabajo, esta conclusión es importante: el simulador de multirrotores de Aerostack2 puede no ofrecer el fotorrealismo de otras plataformas, pero su ejecución sin interfaz gráfica y su integración natural con ROS 2 lo hacen atractivo para construir una infraestructura de entrenamiento conectada con un *stack* robótico real.

En este punto conviene precisar el alcance de la vectorización. El soporte para varios drones en un único simulador compartido y un único proceso Python permite validar la viabilidad funcional de escenarios con N instancias del entorno. En la infraestructura de entrenamiento preparada, la opción prudente es utilizar vectorización síncrona, por ejemplo mediante `DummyVecEnv`, antes de plantear esquemas asíncronos o afirmaciones de aceleración. Sin métricas de rendimiento comparables, la vectorización debe presentarse como capacidad de la infraestructura, no como mejora cuantificada del entrenamiento.

2.3.0.0.1. Herramientas de observación y monitorización. En una infraestructura de aprendizaje por refuerzo integrada con ROS 2, la observación del sistema no

se limita a las métricas internas del algoritmo. RViz2 proporciona una visualización espacial del estado del UAV, de sus referencias y de los marcadores publicados en el grafo ROS 2, por lo que resulta útil para validar cualitativamente que la escena simulada y los comandos robóticos son coherentes. TensorBoard, por su parte, permite inspeccionar las métricas emitidas por Stable-Baselines3 y por los monitores de episodio, como recompensa media, longitud de episodio y señales auxiliares registradas durante la ejecución. En este trabajo, ambas herramientas se emplean como soporte de validación y diagnóstico, no como sustituto de las métricas cuantitativas del entorno.

2.4. Síntesis y posicionamiento del trabajo

La revisión anterior permite extraer varias conclusiones. En primer lugar, el aprendizaje por refuerzo ha demostrado ser una herramienta válida en la literatura para distintos problemas de control de drones, incluyendo estabilización, seguimiento, navegación y vuelo agresivo [Kaufmann et al., 2020; Pi et al., 2021; Yoo et al., 2021; Song et al., 2023]. En segundo lugar, el rendimiento final depende de forma decisiva del nivel de abstracción de la acción, del diseño de la recompensa y de la calidad del simulador empleado [Kaufmann et al., 2022; Chan et al., 2024]. En tercer lugar, muchos trabajos priorizan simuladores especializados o entornos Gym independientes, mientras que la integración reproducible con un ecosistema robótico basado en ROS 2 introduce problemas de ingeniería que también son relevantes para la investigación aplicada [Fernández-Cortizas et al., 2023; Song et al., 2021; Krishnan et al., 2021; Panerati et al., 2021].

A partir de las fuentes revisadas, puede inferirse una oportunidad de trabajo en la construcción de entornos de aprendizaje por refuerzo vectorizables y directamente integrados con un *stack* robótico operativo como Aerostack2. Esta necesidad es más clara cuando el problema de control no se formula a nivel de actuadores, sino a un nivel intermedio próximo a la ejecución de misiones: recibir una referencia relativa al objetivo y generar comandos de velocidad compatibles con la arquitectura del dron, incorporando además una restricción de *path-facing* para alinear el yaw con la dirección de movimiento.

En este punto se sitúa el presente Trabajo Fin de Máster. La contribución diferencial no debe expresarse como una mejora de rendimiento todavía no cuantificada, sino como una integración reproducible entre Gymnasium, Stable-Baselines3, ROS 2 y Aerostack2. El entorno `AS2TestEnv-v0`, la configuración PPO, los scripts de entrenamiento y la validación de vectorización proporcionan la base para ejecutar experimentos posteriores de forma sistemática. La comparación con un controlador PID de referencia queda planteada como parte necesaria de la evaluación, pero solo podrá sostenerse como resultado cuando existan métricas experimentales suficientes.

Desde esta perspectiva, el trabajo se alinea con la literatura reciente sobre control aprendido para cuadricópteros, pero desplaza el foco desde la obtención inmediata de un controlador superior hacia la construcción rigurosa de la infraestructura experimental. Esta decisión es importante porque, sin una interfaz estable, una formulación clara de observaciones y acciones, una recompensa trazable y una estrategia de evaluación reproducible, cualquier comparación posterior entre aprendizaje por refuerzo y control clásico carecería de una base experimental sólida.

Capítulo 3

Diseño del sistema

Este capítulo presenta el diseño del sistema desarrollado para formular y ejecutar un entorno de aprendizaje por refuerzo orientado al control de un UAV en Aerostack2. La descripción se centra en la arquitectura, en la abstracción del problema de control y en las decisiones que permiten integrar Gymnasium con la interfaz robótica de ROS 2. No se aborda aquí la programación detallada de los módulos, sino las decisiones de diseño que estructuran la solución y delimitan su comportamiento.

3.1. Arquitectura general

La arquitectura se organiza como una cadena de responsabilidades que separa el entrenamiento del agente, la definición del entorno y la comunicación con el simulador. En el nivel superior se sitúan los scripts de entrenamiento, encargados de instanciar el entorno y conectar el algoritmo de aprendizaje por refuerzo con la API estándar de Gymnasium. Esta API actúa como frontera entre el agente y el sistema robótico, de modo que el agente no opera directamente sobre ROS 2, sino sobre un contrato de observaciones, acciones, recompensas y condiciones de finalización.

El componente central de esta frontera es la clase `AS2TestEnv`, definida como una especialización de `gym.Env` y registrada como `AS2TestEnv-v0`. Su función de diseño consiste en traducir cada paso del entorno en órdenes comprensibles por Aerostack2 y, a la inversa, convertir el estado relativo del UAV en una observación apta para el agente. La comunicación con Aerostack2 se realiza mediante `as2_python_api.DroneInterface`, mientras que el envío de velocidades se encapsula a través de `SpeedMotion` y del método `send_speed_command_with_yaw_speed`. De este modo, el agente queda desacoplado de los detalles de bajo nivel de ROS 2 y del simulador multirrotor de Aerostack2 [Fernández-Cortizas et al., 2023].

La Figura 3.1 resume esta separación por capas. La interacción avanza desde los scripts de entrenamiento hacia la API Gymnasium, el entorno `AS2TestEnv`, la interfaz `DroneInterface/SpeedMotion`, ROS 2 y, finalmente, el AS2 Multirrotor Simulator. La última capa incorpora, en la versión final del sistema, un overlay con un servicio explícito de reinicio de estado. Esta estructura sigue un criterio de aislamiento. El entrenamiento decide acciones en un espacio matemático acotado, el entorno aplica las transformaciones necesarias y Aerostack2 ejecuta los comandos en el simulador. Además, la inicialización de ROS 2 se diseña como un recurso único por proceso

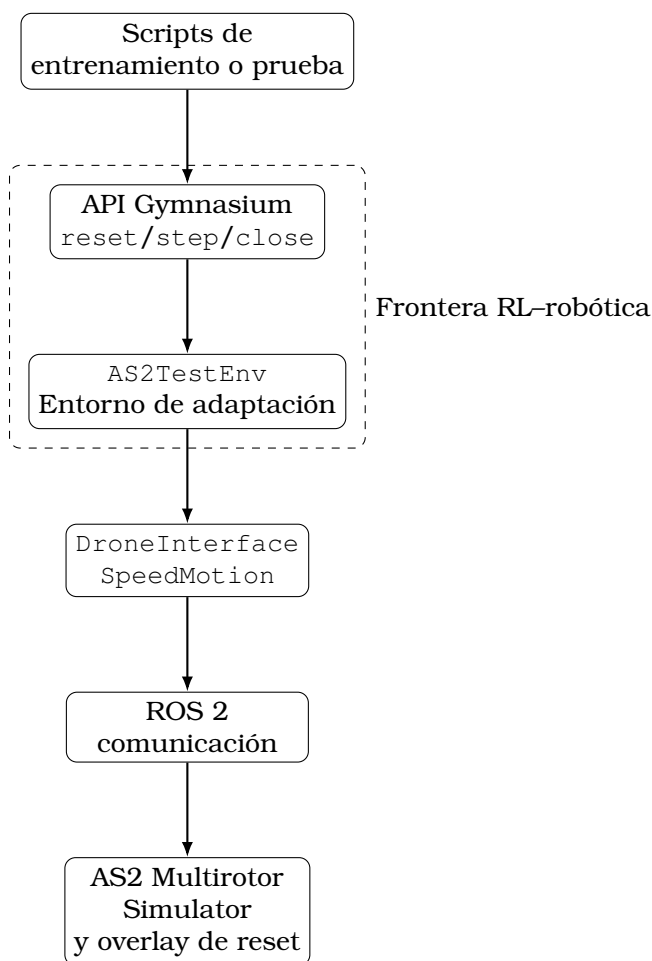


Figura 3.1: Arquitectura general del sistema, separando la lógica de aprendizaje, la interfaz Gymnasium, la comunicación robótica con Aerostack2 y el simulador con servicio de reinicio.

mediante las variables de control `_rcldpy_initialized` y `_rcldpy_lock`, con el fin de evitar inicializaciones repetidas y reducir riesgos de carrera cuando se crean varias instancias del entorno.

La incorporación del servicio de reinicio modifica una hipótesis inicial del diseño. El reinicio deja de depender únicamente de comandos de velocidad o de comportamientos de navegación de Aerostack2, y pasa a contar con una operación propia del simulador para restaurar pose, velocidades, referencias internas, controlador, odometría e IMU. Esta decisión se introduce porque, en aprendizaje por refuerzo, un episodio solo es válido si el estado inicial y la accionabilidad posterior al reset son verificables. El entorno decide cuándo reiniciar, el simulador aplica la mutación física del estado y el propio entorno vuelve a certificar telemetría y movimiento antes de entregar la siguiente transición a PPO.

3.2. Formulación del problema de control

El problema de control se formula como una tarea de navegación local hacia un objetivo relativo. En este contexto, navegación local significa que el agente no reci-

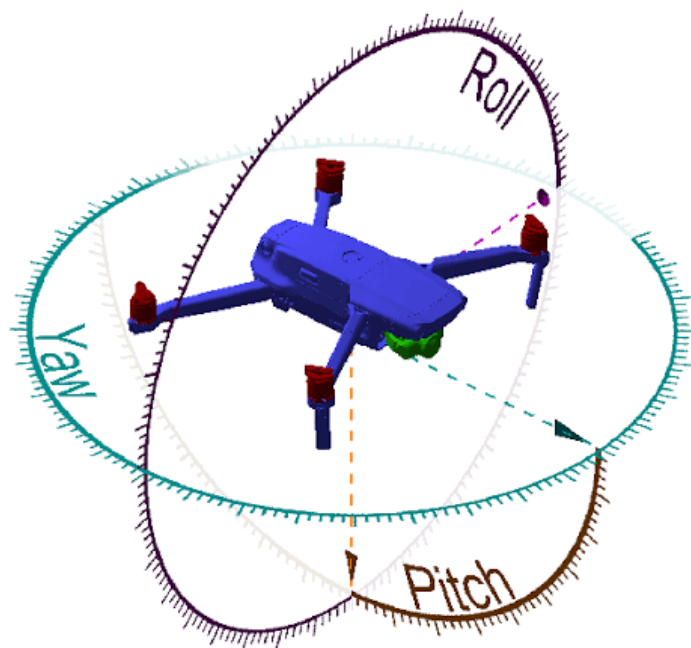


Figura 3.2: Representación conceptual de la pose de un dron mediante los ángulos de *roll*, *pitch* y *yaw*. Fuente: ResearchGate.

be una misión global completa ni una trayectoria predefinida con todos sus puntos intermedios, sino la relación geométrica entre el estado actual del UAV y un objetivo inmediato. A partir de esa relación, el agente debe emitir comandos de velocidad que acerquen el vehículo al objetivo y mantengan una orientación coherente con la dirección de desplazamiento.

Antes de definir formalmente las observaciones y acciones, conviene distinguir las magnitudes básicas que describen el movimiento de un multirrotor. La posición indica dónde se encuentra el UAV en el espacio; la velocidad indica cómo cambia esa posición; y la orientación describe la rotación del vehículo respecto a sus ejes. En un dron se suelen emplear tres ángulos principales, denominados *roll*, *pitch* y *yaw*. El *roll* representa la inclinación lateral, el *pitch* la inclinación hacia delante o hacia atrás, y el *yaw* la rotación alrededor del eje vertical. La Figura 3.2 resume esta idea de pose del dron mediante una representación conceptual tomada de ResearchGate.¹

En el presente trabajo no se controla directamente la actitud completa del dron ni las velocidades de giro de cada rotor. La acción se define a un nivel de abstracción superior, en el que el agente genera velocidades lineales y una velocidad de guiñada. Esta elección sitúa el aprendizaje por refuerzo por encima del control de bajo nivel y delega en Aerostack2 la ejecución de los comandos sobre el simulador. Desde el punto de vista del diseño, esta separación reduce la complejidad del problema aprendido y mantiene la compatibilidad con la interfaz de control de velocidad disponible.

La formulación distingue también entre la dirección frontal del UAV y la dirección real de avance. Un dron puede desplazarse hacia un objetivo aunque su eje frontal

¹Imagen disponible en ResearchGate: https://www.researchgate.net/figure/A-conceptual-representation-of-drone-pose-A-drone-here-represented-by-a-DJI-Mavic-2-is_fig1_356985671. Según la página consultada, la figura aparece con licencia Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International.

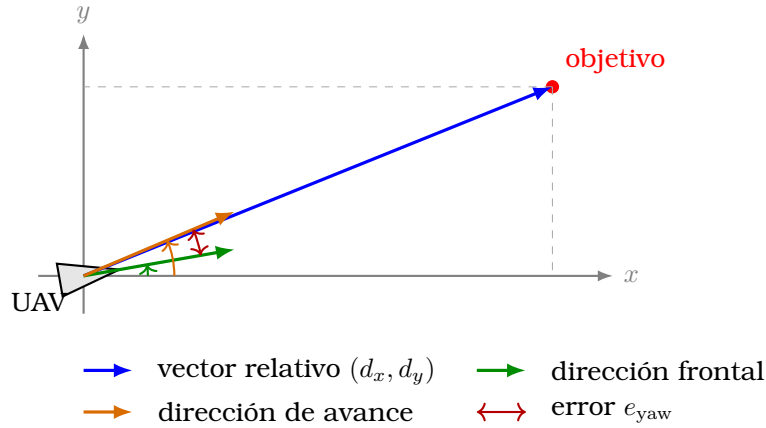


Figura 3.3: Relación geométrica entre el UAV, el objetivo relativo, la dirección de avance y la orientación de guiñada utilizada en el criterio de *path-facing*.

no esté alineado con la trayectoria seguida. Por ello, el diseño inicial introduce el criterio de *path-facing*, con el que se buscaba que la guiñada fuese coherente con la dirección horizontal del movimiento además de reducir la distancia al objetivo. La Figura 3.3 ilustra esta relación entre posición actual, objetivo, vector relativo, dirección de avance y error de orientación.

La acción se define mediante el vector continuo

$$a = [v_x, v_y, v_z, v_{yaw}].$$

Las tres primeras componentes representan velocidades lineales en los ejes cartesianos y la cuarta representa la velocidad angular de guiñada. El espacio de acciones se acota mediante los parámetros `max_vel` y `max_yaw_vel`, lo que introduce una restricción explícita de diseño sobre la magnitud de las órdenes que el agente puede emitir. Esta acotación es necesaria porque una política de aprendizaje por refuerzo puede proponer acciones arbitrarias dentro de su espacio de salida, mientras que el sistema físico o simulado solo debe recibir comandos dentro de un rango admisible.

La observación se define mediante el vector relativo

$$o = [d_x, d_y, d_z, d_{yaw}].$$

Cada componente se normaliza en el intervalo $[-1, 1]$. Las tres primeras componentes codifican el desplazamiento relativo hacia el objetivo, y la cuarta codifica la diferencia relativa de guiñada. La normalización reduce la dependencia respecto a la escala concreta del escenario y proporciona al agente entradas con rangos homogéneos, una condición práctica para estabilizar el tratamiento de las observaciones por parte del algoritmo de aprendizaje.

3.3. Diseño del entorno Gymnasium

El entorno Gymnasium se diseña como una adaptación entre el ciclo estándar `reset-step-close` y las operaciones necesarias para preparar, gobernar y finalizar el vuelo simulado. Esta decisión permite que el agente interactúe con el UAV mediante una

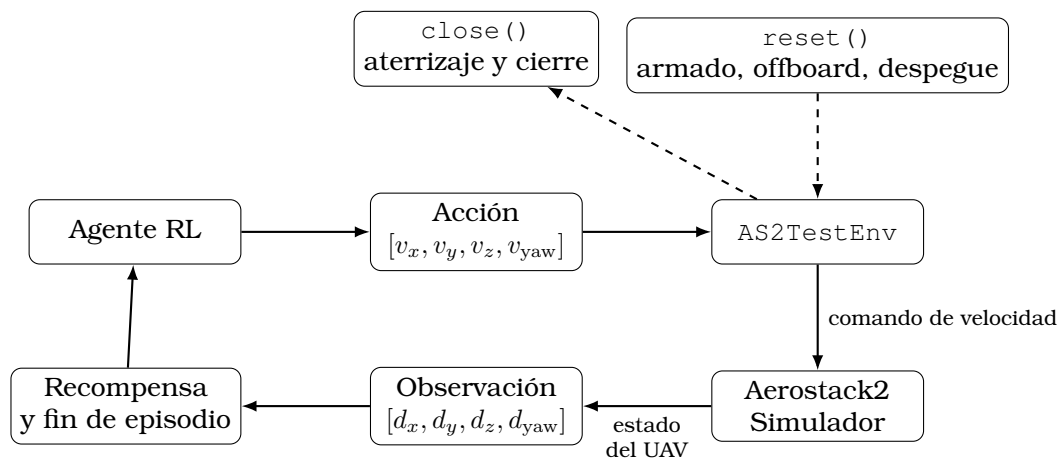


Figura 3.4: Ciclo de interacción entre el agente, el entorno Gymnasium y el simulador durante un episodio.

interfaz conocida en aprendizaje por refuerzo, mientras que la lógica específica de Aerostack2 permanece encapsulada dentro del entorno.

El método `reset` establece el estado inicial de un episodio. Su diseño incluye el armado del UAV, la activación del modo *offboard*, el despegue y la devolución de la primera observación junto con la información auxiliar del episodio. Por tanto, el reinicio no se limita a reinicializar variables internas, sino que prepara al sistema simulado para aceptar comandos de velocidad desde el agente. Esta preparación es relevante porque el contrato de Gymnasium presupone que, tras `reset`, el entorno está listo para ejecutar `step` sin operaciones adicionales por parte del algoritmo.

El método `step` constituye el núcleo de la interacción. En cada llamada, el entorno recibe la acción continua del agente, la transforma en una orden de velocidad enviada mediante `SpeedMotion`, actualiza la observación relativa y calcula la recompensa y los indicadores de fin de episodio. El resultado sigue el contrato de Gymnasium y devuelve observación, recompensa, indicador de terminación, indicador de truncamiento e información auxiliar. La separación entre *terminación* y *truncamiento* permite distinguir un episodio que acaba por alcanzar una condición del problema de otro que finaliza por límite externo de pasos.

El método `close` se diseña como una operación segura de cierre del entorno. Su responsabilidad es aterrizar el UAV y liberar o cerrar los recursos asociados a la comunicación con Aerostack2. Esta decisión busca reducir el riesgo de que el final de un experimento deje el simulador en un estado inconsistente y concentra la lógica de apagado en el mismo componente que gestiona la interacción durante el episodio.

El entorno contempla también un modo de diseño denominado `randomize_hover_start`. Cuando está activo, el inicio, el objetivo y la guiñada del episodio se muestrean dentro de límites configurables y se evitan casos triviales mediante una separación mínima entre inicio y objetivo. Las configuraciones más recientes incorporan además un margen de muestreo respecto a las fronteras de la escena y un límite máximo de distancia inicio-objetivo. Esta decisión evita que las observaciones relativas se saturen en $[-1, 1]$ desde el inicio del episodio y reduce la probabilidad de que un agente no entrenado genere inmediatamente una terminación fuera de límites.

3.4. Diseño del reinicio de episodios y accionabilidad

La Figura 3.4 muestra el ciclo de interacción que se repite durante un episodio. A partir de este ciclo, la Cuadro 3.1 sintetiza el contrato principal que el entorno ofrece al algoritmo de aprendizaje.

Elemento	Diseño adoptado
Observación	Vector relativo normalizado $[d_x, d_y, d_z, d_{yaw}]$
Acción	Velocidades continuas $[v_x, v_y, v_z, v_{yaw}]$
Recompensa	Distancia normalizada al objetivo y bonificación terminal
Finalización	Éxito, salida de límites, baja altitud insegura o truncamiento temporal

Cuadro 3.1: Resumen del contrato principal del entorno Gymnasium.

3.4. Diseño del reinicio de episodios y accionabilidad

El reinicio de episodios constituye una decisión de diseño central en un entorno de aprendizaje por refuerzo para robótica aérea. En un entorno puramente numérico, reiniciar un episodio puede consistir en asignar directamente un nuevo estado inicial. En un simulador robótico integrado con ROS 2 y Aerostack2, en cambio, el reinicio afecta al estado físico del multirrotor, al controlador, a los modos de operación de la plataforma y a la coherencia temporal de las lecturas publicadas. Por este motivo, el diseño del entorno no trata el método `reset` como una operación auxiliar, sino como una parte crítica de la validez experimental.

El diseño contempla varios niveles de reinicio. El nivel básico corresponde al reinicio clásico mediante armado, activación del modo *offboard* y despegue. Este procedimiento es suficiente para una primera inicialización, pero resulta lento y no siempre adecuado entre episodios de entrenamiento. Para experimentos controlados se introduce una pose inicial fija, o `fixed_start_pose`, que separa explícitamente el punto de arranque del objetivo. Cuando el UAV ya se encuentra en vuelo, el retorno a dicha pose se plantea preferentemente mediante comandos de velocidad acotados. Esta decisión evita depender de comportamientos bloqueantes y mantiene la interacción dentro del mismo tipo de órdenes que usa el agente.

También se contempla el reinicio mediante un servicio del simulador. Este mecanismo permite restablecer el estado del multirrotor de forma más directa, pero se considera una herramienta de recuperación controlada y no una solución ciega. Teletransportar un sistema dinámico de un estado terminal a un estado inicial puede generar inconsistencias si los estados internos de la dinámica o del controlador no quedan sincronizados. Por ello, el servicio de reinicio se acompaña de comprobaciones posteriores de pose, modo de operación y capacidad de aceptar comandos físicos antes de declarar válido el inicio de un nuevo episodio.

El servicio de reinicio se diseña como una operación propiedad del simulador y se invoca bajo el espacio de nombres del vehículo, por ejemplo `/drone0/platform/reset_simulator_s`. Su contrato recibe una pose objetivo y tolerancias de validación, y debe devolver evidencia inmediata de la mutación aplicada mediante errores de posición y guiñada, normas residuales de velocidad, aceleración, fuerza y par, y estado de motores. Además, sincroniza la capa de plataforma para que Aerostack2 quede conectado, armado, en modo *offboard* y en estado de vuelo. La respuesta del servicio no se considera por sí sola suficiente para iniciar el siguiente episodio; después del servicio

se comprueba la telemetría publicada y se ejecuta una prueba de accionabilidad con movimiento físico medible.

Una decisión relevante del diseño es que la aceptación de un comando por la API robótica no se considera evidencia suficiente de que la acción sea efectiva. El entorno debe distinguir entre un comando que ha sido aceptado por la interfaz y un comando que produce movimiento físico medible en el simulador. Esta diferencia es crítica para aprendizaje por refuerzo. Si el agente recibe observaciones y recompensas mientras sus acciones no modifican realmente el estado del UAV, los datos recogidos dejan de representar la tarea que se pretende aprender. Por ello, los fallos de accionabilidad se tratan como condiciones de rechazo temprano del entrenamiento y no como episodios válidos.

Estado o fallo	Significado de diseño	Respuesta prevista
reset timeout	El UAV no alcanza la pose inicial dentro del tiempo definido.	Fallo acotado del reinicio y rechazo de continuar el entrenamiento.
post_controller_reach	Tras reiniciar o estabilizar el controlador, no se recupera la tolerancia de posición.	Diagnóstico específico del acoplamiento entre reset y controlador.
post_controller_hold	La pose se alcanza de forma transitoria, pero no se mantiene estable.	Rechazo del reset para evitar datos de episodio físicamente inválidos.
Comando aceptado sin movimiento	La interfaz acepta la orden, pero no se observa desplazamiento físico suficiente.	Fallo cerrado de accionabilidad antes de iniciar o continuar el entrenamiento.
Servicio de reset aplicado sin accionabilidad	El simulador informa que la pose se ha restaurado, pero el camino de comandos posterior no produce desplazamiento suficiente.	Reintento completo del servicio y, si persiste, parada explícita del entrenamiento.

Cuadro 3.2: Estados y fallos considerados en el diseño del reinicio de episodios.

El Cuadro 3.2 resume los principales estados de fallo que guían el diseño. En todos los casos se favorece un fallo explícito y trazable frente a un bloqueo indefinido o a la continuación de un entrenamiento con datos contaminados por un estado físico no válido.

3.5. Diseño de la recompensa y condiciones de finalización

La recompensa final usada en la fase de experimentación principal se diseña para mantener una señal mínima e interpretable, alineada con el criterio del tutor. En cada paso se penaliza únicamente la distancia normalizada al objetivo. Para ello, si d_t representa la distancia euclídea entre el UAV y el objetivo, y $d_{\text{máx}}$ representa la distancia máxima de normalización. A partir de ambas magnitudes se obtiene

$$d_{\text{norm}} = \min\left(\frac{d_t}{d_{\text{máx}}}, 1\right), \quad r_{\text{dist}} = -d_{\text{norm}}. \quad (3.1)$$

En el entorno implementado, $d_{\text{máx}}$ se calcula a partir del límite espacial de posición del escenario, de forma que la distancia normalizada queda acotada en el intervalo

$[0, 1]$.

El entorno conserva la capacidad de calcular términos adicionales, como progreso, seguridad vertical o coherencia de guiñada con la dirección de movimiento. Sin embargo, esos términos se desactivan en Exp010–Exp012 para evitar que el agente optimice una recompensa de shaping distinta de la tarea principal. La guiñada se mantiene como requisito terminal suave. Cuando el UAV alcanza la región objetivo, la bonificación de éxito se reduce en función del error de guiñada. Si e_{yaw} representa el error angular normalizado, la recompensa terminal de éxito se calcula mediante

$$r_{\text{success}} = R_{\text{success}} - w_{\text{yaw}} \frac{|e_{\text{yaw}}|}{\pi}. \quad (3.2)$$

La expresión final aplicada en las configuraciones Exp010–Exp012 incorpora las condiciones terminales del episodio de la forma siguiente.

$$r_t = \begin{cases} -P_{\text{oob}}, & \text{si el UAV sale de los límites del escenario,} \\ -P_{\text{oob}}, & \text{si el UAV entra en baja altitud insegura,} \\ r_{\text{success}}, & \text{si } d_t < d_{\text{threshold}}, \\ -d_{\text{norm}}, & \text{en el resto de pasos.} \end{cases} \quad (3.3)$$

El truncamiento por alcanzar `max_steps` no introduce una recompensa adicional, sino que limita la duración del episodio cuando no se ha alcanzado una condición terminal. El Cuadro 3.3 resume los componentes de esta formulación y su efecto esperado.

3.6. Diseño de la ejecución vectorizada

La ejecución vectorizada se diseña para permitir varias instancias lógicas del entorno sobre un único simulador compartido. El diseño utiliza espacios de nombres diferenciados, `drone0` a `drone3`, de forma que cada UAV simulado pueda ser identificado y gobernado de manera independiente dentro de la misma infraestructura ROS 2. Esta organización reduce la necesidad de duplicar toda la infraestructura del simulador, pero exige tratar con cuidado la inicialización de ROS 2, los nombres de los drones y la sincronización entre episodios.

Desde el punto de vista del diseño de ejecución, `SyncVectorEnv` ofrece un mecanismo directo de ejecución síncrona en el que las instancias avanzan de forma coordinada. En el contexto de `Stable-Baselines3`, el diseño contempla `DummyVecEnv` y `SubprocVecEnv` como alternativas configurables. La opción síncrona, equivalente en este caso a `DummyVecEnv`, constituye el camino prudente cuando se comparte un simulador y se desea minimizar la complejidad de concurrencia. `SubprocVecEnv` puede aportar aislamiento por proceso, pero también introduce mayores exigencias sobre la inicialización de ROS 2 y la gestión de recursos compartidos.

La decisión principal de diseño consiste, por tanto, en priorizar la corrección de la interacción sobre la paralelización agresiva. En un sistema donde varias instancias comparten simulador, una vectorización incorrecta puede producir interferencias entre drones, bloqueos por inicialización repetida o estados de episodio difíciles de atribuir. Por ello, la arquitectura favorece una vectorización controlada, con espacios de

Diseño del sistema

Componente	Expresión	Condición	Efecto esperado
Penalización por distancia	$r_{\text{dist}} = -d_{\text{norm}}$	En cada paso	Favorece que el UAV reduzca progresivamente la distancia al objetivo.
Bonificación por éxito	$R_{\text{success}} - w_{\text{yaw}} e_{\text{yaw}} /\pi$	Si $d_t < d_{\text{threshold}}$	Premia alcanzar la región objetivo y reduce la bonificación si la guiñada terminal es inadecuada.
Penalización por salida de límites	$-P_{\text{oob}}$	Si el UAV supera los límites del escenario	Desincentiva trayectorias inseguras o alejadas de la zona de entrenamiento.
Penalización por baja altitud insegura	$-P_{\text{oob}}$	Si el UAV desciende por debajo del umbral inseguro	Evita que el aprendizaje explote estados cercanos al suelo o físicamente no válidos.
Truncado temporal	Sin recompensa adicional	Si se alcanza el número máximo de pasos	Limita la duración del episodio cuando no se alcanza una condición terminal.

Cuadro 3.3: Componentes de la función de recompensa y condiciones de finalización del entorno.

nombres explícitos y una inicialización de ROS 2 protegida por proceso, antes que asumir incrementos de rendimiento no verificados.

La Figura 3.5 resume esta organización mediante varias instancias lógicas conectadas a una infraestructura de simulación compartida.

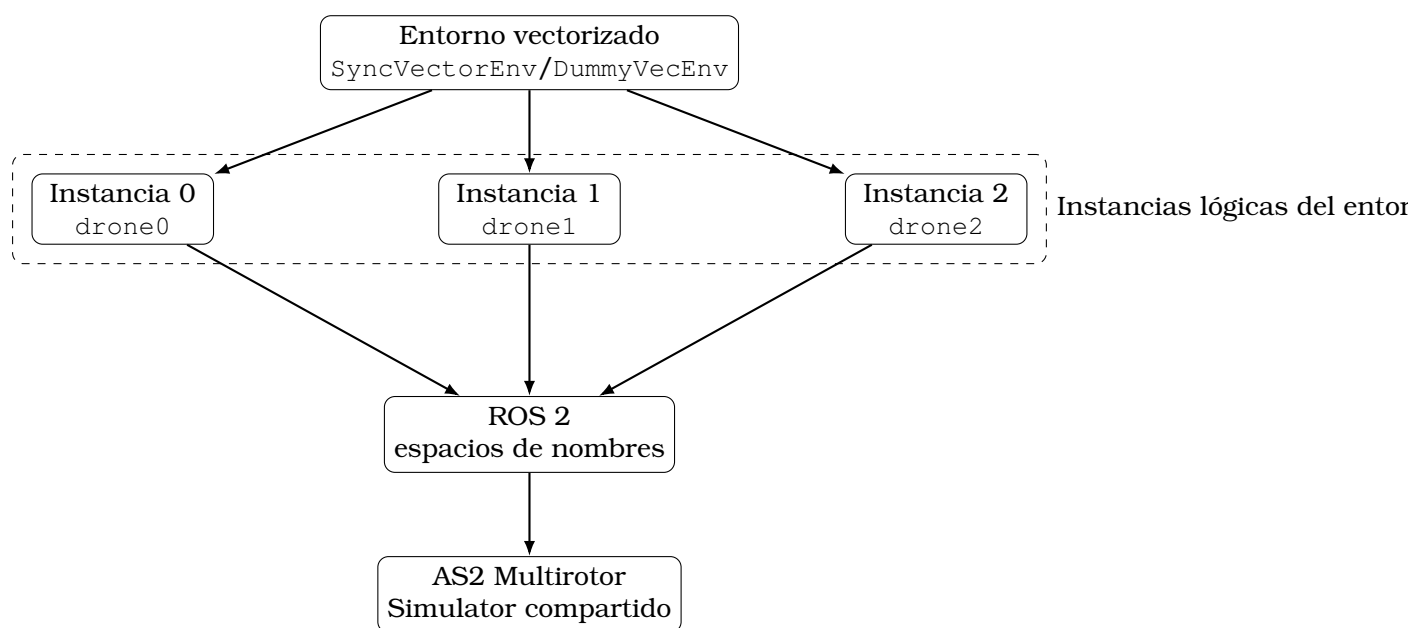


Figura 3.5: Diseño de la ejecución vectorizada mediante varias instancias lógicas asociadas a espacios de nombres diferenciados sobre una infraestructura de simulación compartida.

Capítulo 4

Implementación

Este capítulo describe cómo se materializa en el repositorio el diseño definido en el capítulo anterior. La atención se desplaza desde la formulación del problema hacia la organización del software, la integración efectiva con ROS 2 y Aerostack2, la configuración de PPO y los mecanismos de diagnóstico necesarios para ejecutar experimentos reproducibles.

La implementación no constituye un controlador de referencia clásico. El sistema construye una infraestructura de entrenamiento en la que Aerostack2 conserva el control de bajo nivel, el armado, el despegue y la ejecución simulada, mientras que la política aprendida actúa mediante comandos de velocidad compatibles con la plataforma. Esta distinción evita atribuir a la política responsabilidades que, en la solución desarrollada, pertenecen al *stack* robótico.

4.1. Organización del software

El código desarrollado se distribuye en dos repositorios públicos. El repositorio principal `rl_uav_aerostack2`¹ contiene el entorno de aprendizaje, las configuraciones de entrenamiento, los scripts de ejecución y las utilidades de validación. El repositorio `as2_platform_multirotor_simulator`² contiene el fork del simulador empleado para añadir el servicio de reinicio determinista. Esta separación hace explícito qué parte corresponde al entorno Gymnasium y qué parte modifica la capa física de simulación.

La implementación se organiza alrededor de tres puntos de entrada. El módulo `rl_uav/envs/as2_t` contiene la clase `AS2TestEnv` y concentra la traducción entre Gymnasium y Aerostack2. El fichero `configs/train_ppo.yaml` define de forma declarativa los parámetros del entorno, la vectorización, las rutas de salida y los hiperparámetros de PPO. Por último, `scripts/train_ppo.py` lanza el entrenamiento o valida la configuración mediante un modo *dry-run*. Esta división permite cambiar una configuración experimental sin modificar la lógica del entorno ni los scripts de ejecución.

El repositorio incluye además utilidades de validación para comprobar la conexión monodron, la ejecución vectorizada, el inicio aleatorizado en hover y el comporta-

¹https://github.com/KafeisM/rl_uav_aerostack2

²https://github.com/KafeisM/as2_platform_multirotor_simulator

miento local con *mocks*. Estas comprobaciones no sustituyen a los experimentos, pero evitan confundir errores de infraestructura con errores de aprendizaje. En paralelo, el fork del simulador incorpora el servicio `ResetSimulatorState`, necesario para reiniciar de forma controlada el estado físico del multirrotor durante las campañas experimentales.

Elemento	Responsabilidad principal
<code>rl_uav/envs/as2_test_env.py</code>	Implementación del entorno Gymnasium y puente con Aerostack2.
<code>rl_uav/training/ppo_setup.py</code>	Construcción de entorno vectorizado, rutas de ejecución y modelo PPO.
<code>configs/train_ppo.yaml</code>	Configuración declarativa de entorno, entrenamiento e hiperparámetros.
<code>scripts/train_ppo.py</code>	Punto de entrada para entrenamiento y comprobación de configuración.
<code>scripts/as2_runtime.py</code>	Comprobación y relanzamiento controlado del simulador AS2.
<code>scripts/supervise_train.sh</code>	Supervisor de entrenamientos largos con reinicio del stack AS2 y reanudación desde <i>checkpoint</i> .
<code>as2_platform_multirrotor_sim</code>	Fork del simulador con servicio de reinicio determinista del estado del multirrotor.
<code>as2_sim/launch_sim.bash</code>	Arranque del simulador monodron o multidron mediante nombres de dron.
<code>scripts/test_*</code> <code>scripts/validate_*</code>	y Validaciones locales y reales del entorno, la vectorización y la integración.

Cuadro 4.1: Organización principal del software implementado.

El Cuadro 4.1 resume esta división. Desde el punto de vista de la memoria, lo relevante no es enumerar todos los ficheros del repositorio, sino mostrar que cada bloque tiene una responsabilidad acotada. El entorno define la interacción, la configuración describe el experimento, los scripts ejecutan tareas reproducibles y las utilidades de validación reducen el riesgo de confundir un fallo de simulación con un fallo del algoritmo.

4.2. Integración con Aerostack2 y ROS 2

La integración implementada toma como punto de partida la infraestructura presentada en el Capítulo 2. ROS 2 aporta el grafo de comunicación robótica y Aerostack2 proporciona la capa aérea que encapsula operaciones como armado, modo *offboard*, despegue, aterrizaje y envío de velocidades [Fernández-Cortizas et al., 2023; Open Robotics, 2026]. En este capítulo no se repite su fundamento tecnológico; se describe cómo el entorno `AS2TestEnv` utiliza esas interfaces para convertir acciones de PPO en órdenes ejecutables por el simulador.

La Figura 4.1 muestra la integración implementada. El agente PPO interactúa únicamente con la interfaz Gymnasium. La clase `AS2TestEnv` recibe las acciones, las convierte en comandos de velocidad y utiliza `DroneInterface` y `SpeedMotion` para comunicarse con Aerostack2. A su vez, Aerostack2 utiliza ROS 2 para transportar las órdenes y estados hacia el simulador multirrotor.

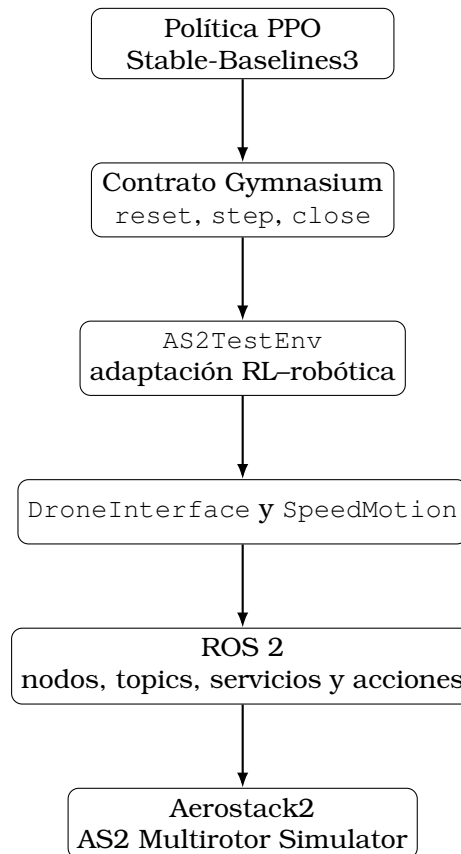


Figura 4.1: Integración implementada entre la política de aprendizaje, la interfaz Gymnasium, Aerostack2 y ROS 2.

En el código, la inicialización de ROS 2 se encapsula dentro de `AS2TestEnv`. El entorno protege la llamada a `rclpy.init()` con un bloqueo y una bandera de clase para evitar inicializaciones repetidas dentro del mismo proceso. Esta decisión es especialmente relevante en escenarios vectorizados, donde varias instancias lógicas del entorno pueden coexistir y compartir recursos de ROS 2. Tras inicializar ROS 2, el entorno crea una instancia de `DroneInterface` asociada al espacio de nombres del dron, por ejemplo `drone0`, y una instancia de `SpeedMotion` para el envío de velocidades.

El método `reset` ejecuta las operaciones necesarias para preparar un episodio. Inicializa la interfaz si aún no existe, arma el dron, activa el modo *offboard*, ejecuta el despegue y genera la primera observación. El método `step` aplica la acción emitida por el agente como un comando de velocidad lineal y velocidad de guiñada mediante `send_speed_command_with_yaw_speed`. Se emplea el marco de referencia *earth*, de forma coherente con la formulación del entorno como navegación local hacia un objetivo relativo. El método `close`, por su parte, realiza un cierre seguro mediante aterrizaje, paso a modo manual y liberación de la interfaz del dron. No se apaga globalmente `rclpy` desde cada instancia, porque hacerlo podría invalidar otras instancias activas del entorno.

El simulador se lanza desde `as2_sim/launch_sim.bash`. Este script permite seleccionar el número de drones y genera nombres de espacio como `drone0`, `drone1`, etc.

Para la ejecución multidron se adopta un criterio de un único simulador compartido con varios espacios de nombres ROS 2, en lugar de iniciar varios simuladores independientes. Esta decisión reduce la sobrecarga de infraestructura y se alinea con la idea de Aerostack2 como marco capaz de operar sistemas aéreos multirrobot [Fernández-Cortizas et al., 2023]. Para hacer más robusta la ejecución, `scripts/as2_runtime.py` comprueba la existencia de las sesiones del simulador y permite relanzarlas antes de ejecutar validaciones que dependen de AS2.

Para los experimentos con reinicio certificado se utiliza un overlay del AS2 Multirrotor Simulator. El servicio añadido se expone bajo el espacio de nombres del vehículo, por ejemplo `/drone0/platform/reset_simulator_state`, y permite aplicar una pose objetivo directamente sobre el estado del simulador. La implementación no se limita a cambiar posición y orientación, sino que también pone a cero velocidades, aceleraciones, fuerzas, pares y estado de motores, reinicia controlador, IMU y odometría, y resincroniza la plataforma de Aerostack2 para dejarla armada, en modo *offboard* y en estado de vuelo. Esta funcionalidad es la base que permite que Exp009 actúe como gate de certificación del reset antes de lanzar experimentos PPO más largos.

4.3. Configuración operativa de PPO

La configuración de PPO se materializa en `configs/train_ppo.yaml`. El Capítulo 2 justifica la elección del algoritmo; esta sección se limita a documentar cómo se parametriza su uso dentro del sistema implementado y cómo se preserva la trazabilidad de cada ejecución.

La configuración se divide en tres bloques principales. El bloque `experiment` define el nombre del experimento, la semilla y el directorio raíz de salida. El bloque `environment` agrupa los parámetros del entorno, incluyendo el identificador `AS2TestEnv-v0`, el número de instancias vectorizadas, el prefijo de espacio de nombres, los límites de posición y velocidad, la duración de cada paso, el objetivo, las condiciones de éxito y las penalizaciones de seguridad. El bloque `ppo` contiene los hiperparámetros del algoritmo, como la tasa de aprendizaje, el número de pasos por actualización, el tamaño de lote, el factor de descuento, el rango de *clipping* y la arquitectura de las redes de política y valor.

Implementación

Parámetro	Valor base	Función
total_timesteps	1000000	Duración máxima de la campaña de entrenamiento.
learning_rate	3.0e-05	Tamaño de actualización de los parámetros de PPO.
n_steps	512	Número de pasos recogidos antes de cada actualización.
batch_size	32	Tamaño de lote usado durante la optimización.
clip_range	0.2	Límite de variación entre política antigua y política nueva.
use_sde	true	Activación de exploración dependiente del estado.
net_arch	[128, 128]	Capas ocultas de las redes de política y valor.

Cuadro 4.2: Principales parámetros PPO definidos en la configuración base de entrenamiento.

El Cuadro 4.2 recoge los parámetros más representativos de la configuración base. Estos valores no deben interpretarse como una optimización definitiva del algoritmo, sino como una configuración inicial trazable y reproducible para validar la infraestructura de entrenamiento. Los capítulos de pruebas y resultados son los que deben determinar si una configuración concreta produce aprendizaje útil, estabilidad suficiente o necesidad de ajuste.

A partir de Exp010 se emplea una configuración más cercana a la especificación experimental indicada por el tutor, con tasa de aprendizaje 3.0e-05, `n_steps=512`, `batch_size=32`, `n_epochs=5`, `ent_coef=0.0`, exploración SDE y redes MLP de dos capas de 128 unidades por rama. Las configuraciones Exp010, Exp011 y Exp012 mantienen estos hiperparámetros y modifican principalmente propiedades del entorno y de la tarea, como la recompensa mínima, la randomización completa, los márgenes de muestreo, el límite de distancia inicio-objetivo, los guards de acción y la reanudación de entrenamiento. Esta separación evita mezclar ajustes del algoritmo con cambios en las condiciones de interacción del UAV.

La configuración también explicita decisiones de ingeniería que afectan a la reproducibilidad. El entrenamiento se fija inicialmente en CPU, coherente con el uso de políticas MLP y con la recomendación habitual de Stable-Baselines3 para evitar sobrecostes de GPU en redes no convolucionales. Además, se separan los directorios de *logs*, monitorización y *checkpoints*, lo que permite analizar una ejecución sin depender únicamente de la salida estándar del proceso. Esta estructura facilita repetir experimentos, comparar configuraciones y aislar fallos entre simulación, entorno y algoritmo.

4.4. Herramientas de visualización y monitorización

La implementación se apoya en herramientas de visualización y monitorización que permiten distinguir entre el funcionamiento del algoritmo, el estado del simulador y la respuesta física del UAV. Esta separación es necesaria porque un entrenamiento

4.4. Herramientas de visualización y monitorización

puede fallar por una configuración de PPO inadecuada, un reinicio inválido del episodio, un problema de comunicación ROS 2 o una orden aceptada por la interfaz que no produce movimiento físico.

RViz2 se utiliza como herramienta de inspección cualitativa del sistema robótico. Aunque el simulador de multirrotores de Aerostack2 no requiere interfaz gráfica, RViz2 permite observar la posición del UAV, la trayectoria reciente y los marcadores publicados para representar el objetivo. Esta visualización no se usa como métrica de aprendizaje, pero sí como apoyo para detectar comportamientos anómalos, como ausencia de movimiento, trayectorias divergentes o incoherencias entre el objetivo configurado y la escena simulada.

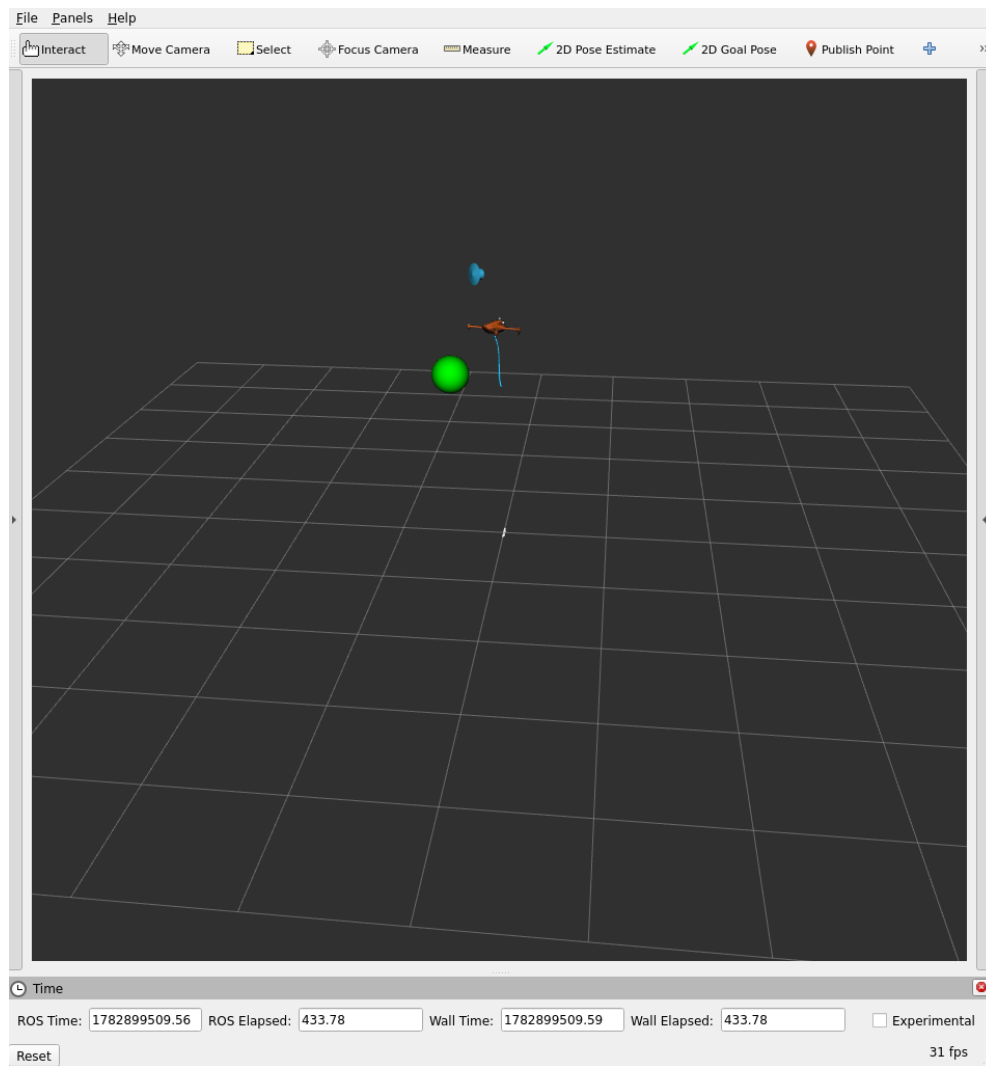


Figura 4.2: Visualización en RViz2 del UAV simulado, la trayectoria reciente y el marcador del objetivo durante una ejecución con Aerostack2.

La Figura 4.2 muestra un ejemplo de la visualización empleada durante las pruebas. El marcador verde representa el objetivo de navegación y el modelo del UAV permite comprobar de forma cualitativa si el movimiento observado es coherente con las acciones enviadas por el entorno.

Implementación

TensorBoard se emplea como herramienta de monitorización de las métricas emitidas por Stable-Baselines3 y por los monitores vectorizados. En las ejecuciones en las que PPO alcanza suficientes pasos para completar una primera actualización, esta herramienta permite inspeccionar magnitudes como recompensa media por episodio, longitud media de episodio y otros escalares registrados. Sin embargo, la interpretación de TensorBoard debe realizarse junto con los ficheros de monitor y los diagnósticos del entorno. Una curva ausente o congelada puede indicar que el entrenamiento no ha alcanzado un *rollout* completo, que el proceso se ha bloqueado durante un reset o que el simulador ha dejado de producir transiciones válidas.

Herramienta	Uso en el trabajo
ROS 2	Comunicación entre nodos, servicios, acciones y flujos de datos del sistema robótico.
Aerostack2	Capa aérea reutilizada para armado, modos de vuelo, control y simulación.
AS2 Multirrotor Simulator	Simulación dinámica del multirrotor usado como fuente de experiencia.
Gymnasium	Interfaz estándar del entorno de aprendizaje por refuerzo.
Stable-Baselines3	Implementación de PPO y utilidades de entrenamiento.
TensorBoard	Monitorización de escalares y evolución de métricas durante ejecuciones PPO.
RViz2	Visualización cualitativa del UAV, el objetivo y la trayectoria en el espacio simulado.

Cuadro 4.3: Herramientas principales utilizadas durante la implementación y validación.

4.5. Implementación del entorno de aprendizaje

La clase `AS2TestEnv` materializa el diseño del Capítulo 3. En lugar de volver a justificar los espacios de observación y acción, esta sección se centra en cómo se ejecuta el contrato en el código. El entorno registra `AS2TestEnv-v0`, define los espacios `Box` correspondientes, aplica los límites configurados y encapsula las operaciones de ciclo de vida que requiere `Aerostack2`.

Durante `reset`, el entorno prepara la interfaz robótica, arma el UAV, activa el modo *offboard*, ejecuta el despegue y devuelve la primera observación normalizada. Cuando se habilita `randomize_hover_start`, la posición inicial, la guiñada y el objetivo se muestrean dentro de los límites configurados. `Exp011` añade márgenes en XY y altura, junto con `max_start_target_distance`, para evitar episodios que comiencen junto a fronteras problemáticas o con la observación saturada desde el primer paso.

Durante `step`, la acción recibida se recorta mediante `max_vel` y `max_yaw_vel`, se envía a `Aerostack2` con `SpeedMotion` y se lee el estado actualizado del UAV. Con esa información se calcula la recompensa, se evalúan las condiciones de terminación y se generan campos de diagnóstico. Las configuraciones `Exp010`–`Exp012` usan una recompensa mínima basada en distancia, bonificación terminal de éxito y penalización por condiciones de seguridad; los términos auxiliares de progreso, *path-facing* y seguridad vertical quedan desactivados para no mezclar el análisis de la tarea con shaping adicional.

4.6. Diagnóstico del reinicio y de la accionabilidad

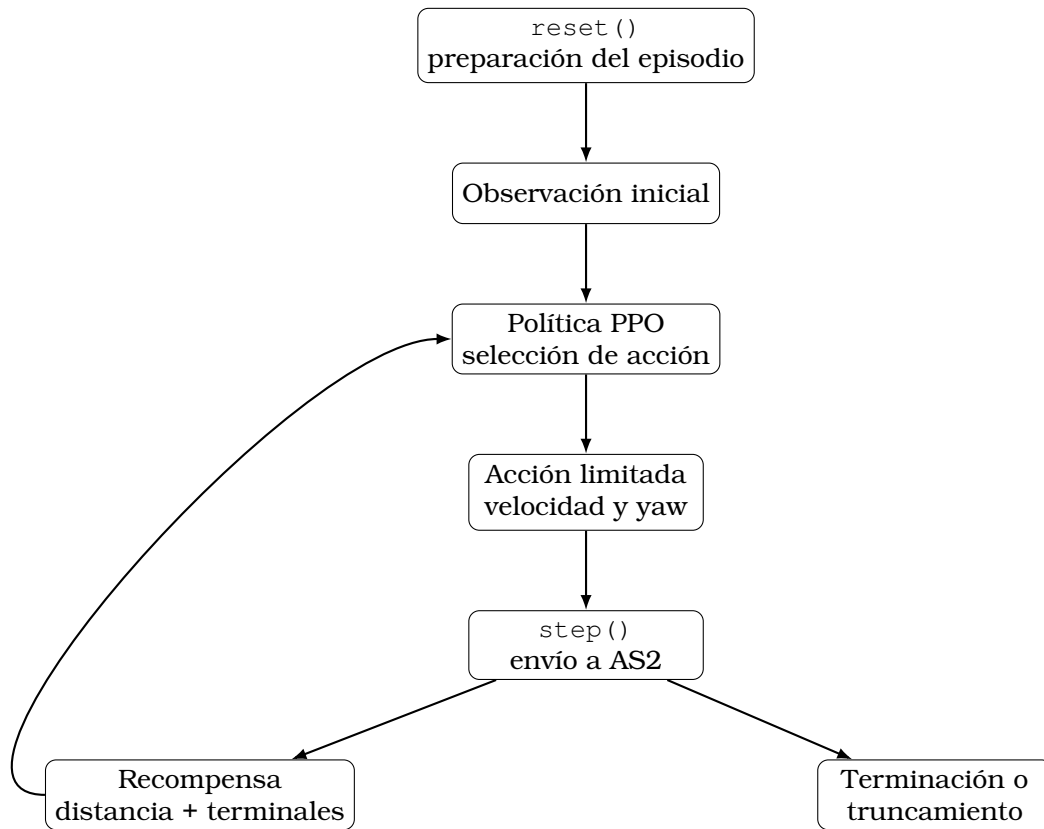


Figura 4.3: Ciclo implementado de un episodio de aprendizaje por refuerzo sobre AS2TestEnv.

La Figura 4.3 resume el flujo implementado. La responsabilidad principal del entorno no es definir de nuevo el problema de aprendizaje, sino convertir cada interacción de PPO en una operación robótica acotada, observable y registrable.

4.6. Diagnóstico del reinicio y de la accionabilidad

La implementación añade instrumentación específica porque el entrenamiento solo es válido si el UAV parte de un estado físico coherente y responde a las acciones enviadas. Por este motivo, el entorno registra el método de reinicio, el camino seguido, el estado del servicio, los errores residuales de posición y guiñada, el desplazamiento físico, la longitud de trayectoria, la aceptación de comandos y la altitud mínima. Estos campos permiten distinguir un fallo de aprendizaje de un fallo previo de simulación o de accionabilidad.

El parámetro `reset_max_vel` separa la autoridad usada por los comandos internos de reinicio de la velocidad máxima permitida a PPO. También se incorpora `reset_yaw_required`, que permite tratar la guiñada como requisito bloqueante o como diagnóstico según la fase experimental. Cuando la interfaz acepta comandos pero el UAV no se desplaza de forma apreciable, el entorno falla de forma cerrada y evita generar experiencia inválida.

El servicio `ResetSimulatorState`, añadido en el fork del simulador, aplica una pose objetivo y reinicia velocidades, aceleraciones, fuerzas, pares, motores, controlador,

IMU y odometría. Sobre ese servicio se implementan reintentos mediante `reset_service_max_attempts` y `reset_service_retry_backoff_s`. Exp010 mostró que abortar tras un fallo transitorio era demasiado frágil; Exp011 añadió reintentos, y Exp012 redujo la frecuencia de reinicios forzados mediante un guard de acción en fronteras.

El guard de Exp012 modifica las acciones que empujan al UAV hacia fuera cuando se aproxima a los límites XY o al techo de la escena. Las terminaciones por salida de límites y baja altitud se mantienen como red de seguridad, pero dejan de ser el mecanismo habitual para corregir a un agente aún no entrenado.

4.7. Infraestructura de entrenamiento y vectorización

La infraestructura de entrenamiento se concentra en `ppo_setup.py`. Este módulo carga la configuración, crea los directorios de salida, instancia los entornos, envuelve la ejecución con `VecMonitor` y construye el modelo PPO. El registro de métricas se realiza sobre el entorno vectorizado para obtener estadísticas de episodio agregadas sin duplicar lógica de monitorización en cada instancia.

La vectorización se implementa mediante espacios de nombres ROS 2. A partir del prefijo `drone` se generan instancias como `drone0`, `drone1`, `drone2` y `drone3`, todas conectadas al mismo simulador compartido. Esta solución evita lanzar un simulador por entorno y mantiene la correspondencia entre instancia de `Gymnasium` y UAV simulado. La implementación puede usar `DummyVecEnv` o `SubprocVecEnv` según la configuración, aunque la validación inicial prioriza ejecuciones controladas antes que paralelización agresiva.

El entrenamiento se ejecuta en dos pasos explícitos. Primero se lanza o verifica el simulador con los scripts de AS2; después se ejecuta `scripts/train_ppo.py` con la configuración seleccionada. Para campañas largas, `train_ppo.py` acepta `-resume-from` y `scripts/supervise_train.sh` automatiza el reinicio del stack AS2, la comprobación del servicio de simulación y la reanudación desde el último *checkpoint*. Esta capa no modifica PPO, pero hace más robusta la ejecución experimental.

La validación inicial confirma que el pipeline puede crear el modelo PPO, conectarse con AS2, enviar comandos de velocidad y generar trazas monitorizables. Esta evidencia prueba la infraestructura de entrenamiento, no el rendimiento final de la política; esa interpretación corresponde a los capítulos de pruebas y resultados.

Capítulo 5

Pruebas y evaluación

Este capítulo define la estrategia de validación del sistema desarrollado. Las pruebas no se limitan a comprobar que el software se ejecuta, sino que permiten valorar si la interfaz entre simulación, *middleware* robótico y entorno de aprendizaje proporciona observaciones, acciones, recompensas y condiciones de reinicio coherentes. En un sistema de aprendizaje por refuerzo para UAV, esta validación previa es imprescindible: si el episodio no se reinicia correctamente o si una acción aceptada por la interfaz no produce movimiento físico, los datos de entrenamiento dejan de representar la tarea que se pretende aprender.

5.1. Estrategia de validación

La estrategia de pruebas se organiza en tres niveles. El primer nivel agrupa pruebas deterministas del entorno ejecutadas sin depender del simulador real. Estas pruebas verifican cálculo de observaciones, recompensas, terminaciones, recorte de acciones y propagación de configuración. El segundo nivel reúne validaciones de integración con Aerostack2 y ROS 2, cuyo objetivo es comprobar que el entorno puede iniciar el UAV, enviar comandos y recibir estado físico del simulador. El tercer nivel corresponde a ejecuciones PPO acotadas, utilizadas para comprobar que la infraestructura de entrenamiento genera métricas, *checkpoints* y diagnósticos interpretables.

La validación se plantea de forma incremental. Antes de lanzar entrenamientos de mayor duración se exige que el entorno pase pruebas de configuración, que el simulador se encuentre en un estado limpio y que los reinicios de episodio sean físicamente válidos. Esta decisión evita confundir un problema de aprendizaje con un problema de infraestructura. Los experimentos que terminan en fallo se mantienen como evidencia útil cuando permiten identificar una condición de diseño o de integración que debe corregirse antes de continuar.

Nivel	Objetivo	Evidencia esperada
Pruebas deterministas	Validar la semántica del entorno sin depender del estado del simulador.	Observaciones, recompensas, terminaciones y parámetros reproducibles.
Pruebas de integración AS2	Verificar conexión, ciclo de vida del UAV, reinicio y comandos de velocidad.	Movimiento físico medible, diagnósticos de reset, servicio de reinicio disponible y ausencia de bloqueos indefinidos.
Ejecuciones acotadas	PPO Comprobar que la infraestructura de entrenamiento produce métricas y artefactos.	Ficheros de monitor, TensorBoard, <i>checkpoints</i> y clasificación de fallos.

Cuadro 5.1: Niveles de validación aplicados al entorno desarrollado.

5.2. Entorno experimental

Las pruebas se realizan sobre el repositorio `rl_uav_aerostack2`, que contiene el entorno `AS2TestEnv-v0`, las configuraciones PPO, los scripts de entrenamiento y las utilidades de validación. La ejecución con simulador se apoya en ROS 2 Humble, Aerostack2 y el AS2 Multirrotor Simulator. La ejecución de los scripts Python se organiza mediante el entorno de ejecución del proyecto, que incluye Gymnasium, Stable-Baselines3 y las dependencias de comunicación con Aerostack2.

La observabilidad de las ejecuciones se apoya en varias fuentes. Los ficheros de monitor registran información por episodio, TensorBoard permite inspeccionar métricas agregadas cuando PPO alcanza suficientes pasos de *rollout*, y RViz2 proporciona una vista cualitativa del estado del UAV, el marcador de objetivo y la trayectoria reciente. Además, el entorno incorpora campos específicos de diagnóstico para reset y accionabilidad, de modo que una ejecución fallida pueda clasificarse sin depender únicamente de la inspección visual.

5.3. Pruebas deterministas y de configuración

Las pruebas deterministas verifican las propiedades del entorno que no requieren un simulador activo. Entre ellas se incluyen la forma y rango de la observación, la acotación de acciones, la propagación de parámetros desde YAML, la lógica de éxito, salida de límites y truncamiento, y la compatibilidad con entornos vectorizados. Este nivel de prueba es necesario porque permite detectar errores de formulación antes de introducir la variabilidad propia de ROS 2 y del simulador físico.

5.4. Pruebas con Aerostack2 y reinicio de episodios

Validador	Propósito	Criterio de aceptación
scripts/test_vectorization	Comprobar contrato del entorno, vectorización, reset y diagnósticos con dobles de san. prueba.	Todas las comprobaciones deterministas pasan.
scripts/test_ppo_config	Verificar que los parámetros YAML llegan al entorno y al modelo PPO.	Los parámetros críticos aparecen en la instancia final.
scripts/test_exp010_reward	Comprobar la recompensa mínima usada en la configuración alineada con el tutor.	La recompensa continua depende de la distancia normalizada y el éxito aplica la penalización terminal de guiñada.
scripts/test_exp011_random	Validar inyectores aleatorio de inicio, objetivo y guiñada con margen de seguridad y límite de distancia.	Las muestras respetan separación mínima, distancia máxima y márgenes de escena.
scripts/test_exp012_train	Verificar la configuración del guard de límites, el corredor vertical y las claves de monitorización.	La configuración de Exp012 conserva la recompensa del tutor y activa las protecciones de frontera.
scripts/test_resume_checkpoint	Validar la resolución de <i>checkpoints</i> para reanudación de PPO.	El entrenamiento puede reanudarse desde una ruta explícita o desde el último <i>checkpoint</i> .
scripts/validate_deterministic	Validar reinicios controlados y coherencia de pose inicial.	El UAV o doble de prueba alcanza el estado inicial esperado.
scripts/validate_exp008_execution	Ejecutar una prueba corta con acción fija o PPO y analizar movimiento físico.	Las acciones aceptadas producen desplazamiento suficiente y métricas coherentes.

Cuadro 5.2: Validadores representativos utilizados durante el desarrollo.

El Cuadro 5.2 resume los validadores más relevantes. La existencia de estas pruebas permite tratar el entrenamiento PPO como una fase posterior a la validación del entorno, no como el único mecanismo para detectar fallos.

5.4. Pruebas con Aerostack2 y reinicio de episodios

Las pruebas con Aerostack2 evalúan si el contrato de Gymnasium puede sostenerse sobre un sistema robótico simulado. En particular, se comprueba que el método `reset` deja el UAV en una pose inicial válida, que `step` envía comandos de velocidad y que `close` no deja referencias activas ni estados bloqueantes. Durante las ejecuciones se detectaron fallos de `reset` que no podían tratarse como simples errores de PPO: episodios que comenzaban fuera de límites, comportamientos bloqueantes de retorno a pose inicial, pérdidas de estabilidad tras recrear el controlador y comandos aceptados sin movimiento físico medible.

Como consecuencia de estos fallos, el reinicio de episodios se trasladó parcialmente al simulador mediante un servicio específico del fork del AS2 Multirrotor Simulator. Este servicio recibe la pose objetivo del UAV, muta el estado interno de la dinámica, limpia velocidades, aceleraciones, referencias y estado de actuadores, y sincroniza la capa de plataforma de Aerostack2 para que el vehículo quede armado, en modo *off-board* y en estado de vuelo. La validación del servicio distingue entre dos fronteras de prueba: la respuesta inmediata del servicio, que informa errores de posición, guiñada y normas dinámicas residuales, y la telemetría posterior publicada por ROS 2, que debe confirmar que no aparecen picos transitorios tras el reinicio.

Para evitar que estos casos contaminen el entrenamiento, el entorno registra razones de fallo y adopta una política de fallo acotado. Si el UAV no vuelve a la pose inicial dentro del tiempo configurado, si no se mantiene estable tras el reset o si una acción aceptada no produce movimiento observable, la ejecución se rechaza como evidencia de entrenamiento válido. Este criterio es más estricto que permitir que PPO continúe acumulando pasos, pero resulta necesario para mantener la validez de los datos.

Fallo observado	Causa probable	Mitigación incorporada	Evidencia
Éxitos triviales de un paso	Objetivo demasiado cercano al despegue inicial.	Objetivos desplazados y separación explícita entre inicio y objetivo.	Exp001–Exp002.
Bucle de episodios fuera de límites	Reset fijo insuficiente tras una salida de escena.	<code>fixed_start_pose</code> y entorno controlado a pose inicial.	Exp002–Exp003.
Bloqueo por <code>go_to</code>	Comportamiento de Aerostack2 ejecutándose indefinidamente.	Eliminación del fallback bloqueante y fallo explícito.	Exp006–Exp007.
Timeout de reset por velocidad	Derivas grandes o recuperación insuficiente entre episodios.	Diagnósticos de reset, <code>reset_max_vel</code> y fases de corrección.	Exp007–Exp008a.
Comando aceptado sin movimiento	Desacoplamiento entre aceptación de API y acción física tras reset.	Prueba de accionabilidad y fallo cerrado antes de entrenar.	Smokes Exp008a.
Degradación por reinicios de servicio repetidos	Muchos episodios terminan en fronteras no observables y fuerzan teleports sucesivos.	Reintentos completos del servicio, guard de acción en límites y supervisor con reanudación.	Exp010–Exp012.

Cuadro 5.3: Fallos de integración identificados y mecanismos de control incorporados.

Además de las pruebas ejecutadas desde el repositorio del entorno, el fork del simulador incorpora pruebas unitarias sobre el servicio de reinicio. Estas pruebas verifican que el servicio está disponible, que rechaza entradas no finitas o tolerancias inválidas, que limpia referencias dinámicas obsoletas y que mantiene una altitud válida incluso cuando el estado previo del vehículo no era apto para iniciar un nuevo episodio. Estas comprobaciones son relevantes porque el servicio no solo debe modificar la pose externa del UAV, sino también dejar el estado interno del simulador en condiciones compatibles con el siguiente comando de velocidad.

5.5. Métricas y criterios de aceptación

Las métricas empleadas combinan indicadores de aprendizaje e indicadores de infraestructura. En el primer grupo se encuentran la recompensa por episodio, la longitud de episodio, la tasa de éxito y la distancia final al objetivo. En el segundo grupo se registran razones de terminación, desplazamiento físico, longitud de trayectoria, altitud mínima, ratio de comandos aceptados, método de reset y clase de fallo. Esta separación permite determinar si una ejecución fracasa porque la política todavía no resuelve la tarea o porque el entorno no está entregando transiciones físicamente válidas.

Las curvas de TensorBoard se utilizan como evidencia complementaria, no como única prueba de aprendizaje. En particular, las métricas `rollout/ep_rew_mean` y `rollout/ep_len_mean` permiten observar tendencias agregadas, mientras que los campos de monitor por episodio permiten explicar esas tendencias mediante razones de terminación, distancia final, éxitos, salidas de límites o eventos de baja altitud. Esta combinación es necesaria porque una curva plana de recompensa puede deberse a una política que no aprende, pero también a episodios dominados por terminaciones de seguridad o por reinicios costosos del simulador.

El criterio de aceptación para un entrenamiento no se reduce a que el proceso de Python siga ejecutándose. Una ejecución solo se considera apta para análisis de aprendizaje si el entorno inicia episodios desde estados válidos, si las acciones producen movimiento físico medible, si los fallos de reset no se ocultan como episodios válidos y si las métricas de monitorización permiten reconstruir lo ocurrido. En caso contrario, el resultado se documenta como evidencia de integración y se utiliza para mejorar el entorno antes de extraer conclusiones sobre PPO.

Capítulo 6

Resultados

Este capítulo presenta los resultados obtenidos mediante la estrategia de pruebas definida previamente. La exposición distingue entre resultados de infraestructura y resultados de aprendizaje. Esta distinción es esencial en el presente trabajo: la capacidad de lanzar PPO y generar métricas no implica por sí misma que se haya obtenido una política robusta, especialmente cuando los experimentos revelan problemas de reinicio, accionabilidad o coherencia física del simulador.

6.1. Validación funcional de la infraestructura

Las primeras pruebas confirman que la infraestructura básica es operativa. El entorno puede instanciarse como entorno Gymnasium, conectarse con Aerostack2, armar el UAV, activar el modo *offboard*, ejecutar despegue, enviar comandos de velocidad y registrar métricas mediante los mecanismos de monitorización. Esta validación funcional demuestra que la integración entre Gymnasium, Stable-Baselines3, ROS 2 y Aerostack2 es viable.

No obstante, los resultados también muestran que la validez de un entrenamiento depende de condiciones adicionales. En las primeras ejecuciones se observaron éxitos triviales cuando el objetivo estaba demasiado cerca de la pose de despegue, ausencia de escalares de TensorBoard cuando PPO no alcanzaba suficientes pasos de *rollout*, y bucles de episodios fuera de límites cuando el reset no garantizaba un estado inicial seguro. Las iteraciones posteriores permitieron certificar el reinicio mediante el servicio del simulador y ejecutar entrenamientos más largos, pero también revelaron que las salidas de límites y la degradación acumulada por reinicios de servicio podían dominar el comportamiento observado. Estos resultados condujeron a separar el problema en dos niveles: primero validar el ciclo físico de episodio y después analizar el aprendizaje.

6.2. Matriz de experimentos PPO

El Cuadro 6.1 resume la evolución de los experimentos principales. La tabla no debe interpretarse como una búsqueda lineal de hiperparámetros, sino como una secuencia de validación en la que cada ejecución revela una propiedad de la infraestructura.

6.3. Resultados de entrenamiento y monitorización

Experimento	Objetivo	Resultado observado	Interpretación técnica
Exp001	Smoke PPO con configuración inicial.	Episodios de éxito en un paso y métricas insuficientes.	El objetivo estaba demasiado cerca del estado inicial; no era una tarea de navegación útil.
Exp002	Objetivo desplazado y TensorBoard en prueba corta.	Primeros episodios útiles, después bucles de salida de límites.	El pipeline generaba artefactos, pero el reset no garantizaba inicio seguro tras OOB.
Exp003	Introducir <code>fixed_start_pose</code> .	Entrenamiento corto completado con monitor, TensorBoard y <i>checkpoints</i> .	El reset determinista resolvió el bucle inmediato de OOB en una prueba acotada.
Exp004	Target más lejano y marcador RViz.	Se mejoró la observabilidad del objetivo y de métricas.	RViz y marcadores ayudan a validar cualitativamente la escena, no sustituyen métricas.
Exp005	Target estable a 2 m y reset en vuelo.	Primeras evidencias de éxito real hacia un punto, con incidencias de reset.	El reset en vuelo evitó bloqueos de aterrizaje, pero la robustez seguía siendo limitada.
Exp006	Mejorar estabilidad y evitar fallback problemático.	Se detectó bloqueo asociado a <code>go_to</code> .	El retorno a pose inicial debía evitar comportamientos bloqueantes.
Exp007	Gate largo tras eliminar <code>go_to</code> .	Fallo acotado por timeout de reset por velocidad.	El entrenamiento ya no quedaba bloqueado, pero el reset seguía sin ser fiable para runs largos.
Exp008	Reset más robusto con target de 2 m.	Persisten timeouts de reset.	La dificultad seguía siendo alta para depurar aprendizaje si el reset fallaba.
Exp008a	Curriculum de 0,5 m y diagnóstico ampliado.	Mejora parcial, pero aparecen bloqueos de reset y accionabilidad.	El cuello de botella pasa de PPO a la interacción reset-controlador-simulador.
Exp009	Primer gate PPO sobre el stack de reset certificado.	Ejecución de 5102 pasos de monitor, 126 episodios y finalización correcta del proceso.	Se valida la infraestructura de episodio; la tasa de éxito observada, 21/126 episodios, no se interpreta todavía como política robusta.
Exp010	Recompensa mínima alineada con el tutor, randomización completa e hiperparámetros revisados.	Se registran 12852 pasos y 427 episodios antes del fallo; 359 terminan fuera de límites y 65 por baja altitud.	El diseño de recompensa queda alineado, pero las fronteras no observables y el coste de teleport dominan la experiencia.
Exp011	Muestreo en área segura y límite de distancia inicio-objetivo para evitar saturación de observaciones.	Se registran 21347 pasos y 637 episodios; 512 terminan fuera de límites, 121 por baja altitud y solo 3 en éxito.	La saturación inicial se corrige parcialmente, pero el problema estructural de terminaciones y reinicios repetidos persiste.
Exp012	Guard de acción en límites, corredor vertical ampliado, reanudación desde <i>checkpoint</i> y supervisor.	Experimento en ejecución durante la redacción de esta versión.	La hipótesis evaluada es reducir reinicios de servicio y aislar mejor el aprendizaje de PPO de los fallos de infraestructura.

Cuadro 6.1: Resumen de experimentos principales y su interpretación.

6.3. Resultados de entrenamiento y monitorización

Los experimentos muestran que la infraestructura de entrenamiento produce artefactos analizables cuando alcanza suficiente número de pasos: ficheros de monitor, eventos de TensorBoard, *checkpoints* y modelos finales en pruebas acotadas. En particular, las ejecuciones iniciales permiten comprobar que TensorBoard y los monitores de episodio funcionan cuando PPO completa *rollouts* suficientes, mientras que Exp009 confirma que el stack de reset certificado puede sostener una ejecución PPO completa de 5000 pasos. Sin embargo, la presencia de estos artefactos no implica aprendizaje estable: debe analizarse junto con las razones de terminación, la longitud de episodio, la distancia final, la tasa de éxito y los diagnósticos de reset.

Las métricas agregadas de TensorBoard son especialmente útiles para interpretar

Exp010 y Exp011, porque permiten observar si la recompensa media, la longitud de episodio o la tasa de éxito muestran una tendencia temporal. En las condiciones registradas, la evidencia disponible no debe presentarse como una política final aprendida, sino como una secuencia de diagnóstico: Exp010 muestra que la formulación de recompensa e hiperparámetros ya está alineada con el criterio del tutor, pero que la experiencia queda dominada por salidas de límites; Exp011 reduce la saturación de observaciones mediante un área de muestreo más segura, pero no elimina el predominio de terminaciones no deseadas. Por ello, las curvas de TensorBoard deben leerse junto con los ficheros de monitor y con la matriz de experimentos, no de forma aislada.

La comparación cuantitativa entre Exp010 y Exp011 muestra una mejora parcial, pero insuficiente. En Exp010 se registran 427 episodios, con 359 terminaciones por salida de límites, 65 por baja altitud insegura y 3 éxitos. La recompensa media por episodio es aproximadamente $-31,36$, la longitud media es de 30,10 pasos y la distancia final media al objetivo es de 5,41 m. En Exp011 se registran 637 episodios, con 512 salidas de límites, 121 terminaciones por baja altitud, 3 éxitos y un episodio por límite de pasos. La recompensa media mejora hasta aproximadamente $-27,34$, la longitud media aumenta a 33,51 pasos y la distancia final media baja a 3,71 m. Sin embargo, la tasa de éxito permanece por debajo del 1 % en ambos casos y cae a cero en los tramos finales de las curvas de TensorBoard. Por tanto, Exp011 no demuestra aprendizaje robusto, sino que reduce algunos síntomas de Exp010 sin eliminar la causa principal: la política sigue interactuando con límites que no forman parte explícita de la observación.

La evolución de los experimentos no permite afirmar todavía que se haya obtenido una política PPO robusta para navegación estable. Sí permite afirmar que se ha construido una infraestructura capaz de revelar por qué un entrenamiento no es válido. Las ejecuciones detectaron éxitos triviales, episodios contaminados por reinicios inválidos, bloqueos del comportamiento `go_to`, timeouts de reset por velocidad, comandos aceptados que no se traducían en desplazamiento físico suficiente y degradación asociada a un número elevado de teleports del simulador. Estos resultados desplazan la discusión desde la optimización de PPO hacia la validez del entorno de entrenamiento.

6.4. Análisis de fallos de integración

El resultado más relevante desde el punto de vista de ingeniería es la identificación del reinicio y de la accionabilidad como cuellos de botella. En un entorno de aprendizaje por refuerzo, cada episodio debe comenzar en un estado bien definido y cada acción debe modificar el estado de forma coherente. Cuando el UAV no regresa a la pose inicial, cuando el controlador no mantiene la pose tras el reset o cuando la API acepta comandos que no producen movimiento, el entrenamiento deja de generar experiencia válida.

La solución adoptada consiste en hacer visibles esos fallos y detener la ejecución antes de acumular datos no representativos. Esta política reduce el número de ejecuciones largas aparentemente productivas, pero aumenta la calidad de la evidencia experimental. En las condiciones evaluadas, el problema principal no se encuentra en la llamada estándar a `model.learn` de Stable-Baselines3, sino en garantizar que

el entorno robótico entregue transiciones físicamente accionables después de cada reinicio.

La incorporación del servicio de reinicio del simulador resolvió una parte fundamental del problema, al permitir teletransportar el UAV a una pose inicial controlada y reinicializar estado dinámico, referencias y sincronización de la plataforma. Sin embargo, Exp010 y Exp011 muestran que un servicio de reset correcto no basta si la tarea genera demasiadas terminaciones en fronteras que la política no observa directamente. Esta observación justifica la introducción en Exp012 de guards de acción en los límites de la escena: el objetivo no es ocultar fallos de seguridad, sino impedir que un agente no entrenado convierta los límites invisibles en una fuente dominante de reinicios costosos.

6.5. Discusión de limitaciones

Los resultados deben interpretarse con varias limitaciones. En primer lugar, las ejecuciones disponibles combinan gates de infraestructura y entrenamientos parciales, no una campaña final de entrenamiento cerrada. En segundo lugar, la comparación con una configuración PID queda planteada como evaluación posterior, ya que no sería metodológicamente correcto comparar un controlador de referencia contra una política PPO cuyo entrenamiento estable aún depende de reducir terminaciones espurias y reinicios costosos. En tercer lugar, los resultados dependen del estado del simulador y de la correcta sincronización entre plataforma, controlador y servicios de reset.

Estas limitaciones no invalidan el trabajo realizado. Al contrario, delimitan con precisión qué parte de la infraestructura está validada y qué parte requiere investigación adicional. La contribución experimental del trabajo reside en haber construido una base reproducible y en haber identificado condiciones de fallo que deben resolverse antes de extraer conclusiones sobre el rendimiento final de PPO.

Capítulo 7

Impacto del trabajo

Este capítulo analiza el impacto potencial del trabajo desde una perspectiva técnica, académica y social. El análisis se centra en las posibilidades que ofrece un entorno de aprendizaje por refuerzo reproducible para UAVs, así como en sus limitaciones y riesgos cuando se traslada desde simulación hacia sistemas robóticos reales.

7.1. Impacto técnico y académico

7.2. Impacto social e industrial

7.3. Consideraciones éticas y de seguridad

7.4. Relación con los Objetivos de Desarrollo Sostenible

Capítulo 8

Conclusiones y líneas futuras

Este capítulo resume las principales conclusiones del trabajo y valora el grado de cumplimiento de los objetivos planteados. La discusión se apoya en la evidencia obtenida durante el diseño, la implementación y las pruebas, evitando formular conclusiones que no estén respaldadas por resultados.

8.1. Conclusiones

El presente trabajo ha desarrollado una infraestructura operativa para formular un problema de control de UAV como entorno de aprendizaje por refuerzo integrado con Aerostack2. El entorno implementado proporciona el contrato estándar de Gymnasium, transforma acciones continuas en comandos de velocidad, calcula observaciones relativas al objetivo y registra información auxiliar suficiente para analizar episodios, recompensas y condiciones de fallo. Con ello se cumple el objetivo de construir una base reproducible sobre la que ejecutar pruebas de aprendizaje y validación.

También se ha validado la integración entre Stable-Baselines3, ROS 2 y el AS2 Multirotor Simulator. Las pruebas muestran que el pipeline puede crear modelos PPO, conectarse al simulador, enviar comandos, generar métricas de monitorización y producir artefactos como eventos de TensorBoard y *checkpoints*. Sin embargo, esta validación funcional no debe confundirse con una demostración de aprendizaje robusto. Los experimentos disponibles indican que la estabilidad del ciclo de episodio es una condición previa para interpretar cualquier curva de entrenamiento.

Una conclusión central es que el principal cuello de botella detectado no es la formulación básica de PPO, sino la integración física del ciclo `reset-step` en un simulador robótico. Los experimentos revelaron éxitos triviales por mala configuración inicial, bucles de salida de límites, bloqueos por comportamientos de retorno, timeouts de reset por velocidad y casos en los que los comandos eran aceptados por la interfaz pero no producían movimiento físico suficiente. Estos hallazgos justifican la instrumentación de reset, la separación entre velocidad de política y velocidad interna de reset, y la comprobación explícita de accionabilidad.

Desde el punto de vista metodológico, el trabajo muestra la importancia de tratar los resultados negativos como evidencia de ingeniería. Un experimento que falla de forma acotada, registra la causa y evita continuar con datos inválidos aporta más información que una ejecución larga cuyo estado físico no puede reconstruirse. Por tanto, la

contribución principal del trabajo es una base de entrenamiento y diagnóstico que permite avanzar hacia aprendizaje robusto con criterios de validez más estrictos.

8.2. Líneas futuras

La primera línea futura consiste en completar la validación de estados, recompensas y terminaciones en coordenadas límite del escenario. Esta validación debe comprobar de forma sistemática que el entorno devuelve observaciones, recompensas y señales de fin de episodio coherentes para posiciones seguras, casos de éxito, salidas de límites y alturas críticas. Este paso permitiría cerrar la validación funcional antes de lanzar campañas de entrenamiento más largas.

Una segunda línea futura es mejorar el mecanismo de reset del simulador. El servicio de reinicio debe garantizar no solo la pose externa del UAV, sino también la coherencia de los estados internos de la dinámica y del controlador. En sistemas dinámicos, cambiar de un estado terminal a un estado inicial en tiempo nulo puede generar inconsistencias si no se reinician correctamente las variables internas. Resolver este punto es necesario para que los episodios de aprendizaje sean independientes y reproducibles.

Otra continuación natural consiste en estudiar una vía de entrenamiento más cercana al simulador físico interno, reduciendo la dependencia de ROS 2 en el bucle de aprendizaje. Una arquitectura basada en *bindings* Python y un paso temporal fijo permitiría integrar la dinámica con un Δt controlado y ejecutar el entrenamiento a mayor velocidad. En ese escenario, ROS 2 quedaría reservado para el despliegue o para validaciones integradas, mientras que la generación masiva de experiencia se realizaría sobre una interfaz más directa y determinista.

Finalmente, una vez estabilizado el ciclo de reset y accionabilidad, el entorno podrá utilizarse para comparar políticas PPO entrenadas frente a controladores de referencia, ampliar el curriculum de navegación, introducir trayectorias más complejas y evaluar transferencia progresiva hacia escenarios multirrobot o plataformas reales. Estas líneas futuras continúan directamente la infraestructura desarrollada y evitan extraer conclusiones de rendimiento antes de garantizar la validez de los datos de entrenamiento.

Bibliografía

- AltexSoft (2021). *Reinforcement Learning Explained: Overview, Comparisons and Applications in Business*. URL: <https://www.altexsoft.com/blog/reinforcement-learning-explained-overview-comparisons-and-applications-in-business/> (visitado 28-06-2026).
- Azar, Ahmad Taher, Anis Koubaa, Nada Ali Mohamed, Habiba A. Ibrahim, Zahra Fathy Ibrahim, Muhammad Kazim, Adel Ammar, Bilel Benjdira, Alaa M. Khamis, Ibrahim A. Hameed y Gabriella Casalino (2021). «Drone Deep Reinforcement Learning: A Review». En: *Electronics* 10.9, pág. 999. DOI: 10.3390/electronics10090999. URL: <https://doi.org/10.3390/electronics10090999>.
- Chan, Jun Hoong, Kai Liu, Yu Chen, A. S. M. Sharifuzzaman Sagar y Yong-Guk Kim (2024). «Reinforcement Learning-based Drone Simulators: Survey, Practice, and Challenge». En: *Artificial Intelligence Review* 57.10, pág. 281. DOI: 10.1007/s10462-024-10933-w. URL: <https://doi.org/10.1007/s10462-024-10933-w>.
- Chollet, Gabriel (2021). *What is ROS*. URL: https://medium.com/@Gabriel_Chollet/what-is-ros-c38493fe3eca (visitado 28-06-2026).
- Eschmann, Jonas, Dario Albani y Giuseppe Loianno (2024). «Learning to Fly in Seconds». En: *IEEE Robotics and Automation Letters* 9.7, págs. 6336-6343. DOI: 10.1109/LRA.2024.3396025. URL: <https://doi.org/10.1109/LRA.2024.3396025>.
- Farama Foundation (2026). *Gymnasium Documentation: Env API*. URL: <https://gymnasium.farama.org/api/env/> (visitado 25-06-2026).
- Fernández-Cortizas, Miguel, Martín Molina, Pedro Arias-Perez, Rafael Perez-Segui, David Perez-Saura y Pascual Campoy (2023). «Aerostack2: A Software Framework for Developing Multi-robot Aerial Systems». En: *arXiv preprint arXiv:2303.18237*. DOI: 10.48550/arXiv.2303.18237. URL: <https://arxiv.org/abs/2303.18237>.
- Kaufmann, Elia, Leonard Bauersfeld y Davide Scaramuzza (2022). «A Benchmark Comparison of Learned Control Policies for Agile Quadrotor Flight». En: *2022 International Conference on Robotics and Automation (ICRA)*, págs. 10504-10510. DOI: 10.1109/ICRA46639.2022.9811564. URL: <https://doi.org/10.1109/ICRA46639.2022.9811564>.
- Kaufmann, Elia, Antonio Loquercio, Rene Ranftl, Matthias Müller, Vladlen Koltun y Davide Scaramuzza (2020). «Deep Drone Acrobatics». En: *Proceedings of Robotics: Science and Systems*. Corvallis, Oregon, USA. DOI: 10.15607/RSS.2020.XVI.040. URL: <https://www.roboticsproceedings.org/rss16/p040.html>.
- Krishnan, Srivatsan, Behzad Boroujerdian, William Fu, Aleksandra Faust y Vijay Janapa Reddi (2021). «Air Learning: a Deep Reinforcement Learning Gym for Autonomous Aerial Robot Visual Navigation». En: *Machine Learning* 110, págs. 2501-2540. DOI: 10.1007/s10994-021-06006-6. URL: <https://doi.org/10.1007/s10994-021-06006-6>.

- Molina, Martín (2025a). «Actor Critic Policy Optimization». Material docente de la asignatura Artificial Neural Networks and Deep Learning, Máster Universitario en Inteligencia Artificial, Universidad Politécnica de Madrid.
- (2025b). «Introduction to Reinforcement Learning». Material docente de la asignatura Artificial Neural Networks and Deep Learning, Máster Universitario en Inteligencia Artificial, Universidad Politécnica de Madrid.
 - (2025c). «Proximal Policy Optimization». Material docente de la asignatura Artificial Neural Networks and Deep Learning, Máster Universitario en Inteligencia Artificial, Universidad Politécnica de Madrid.
- Objectways (2025). *Deep Learning Vs Reinforcement Learning: Key Differences*. URL: <https://objectways.com/blog/deep-learning-vs-reinforcement-learning-key-differences/> (visitado 28-06-2026).
- Open Robotics (2026). *ROS 2 Humble Documentation: Basic Concepts*. URL: <https://docs.ros.org/en/humble/Concepts.html> (visitado 25-06-2026).
- OpenAI (2026a). *ChatGPT*. URL: <https://chatgpt.com/> (visitado 02-07-2026).
- (2026b). *OpenAI Codex*. URL: <https://openai.com/codex/> (visitado 02-07-2026).
- Panerati, Jacopo, Hehui Zheng, Siqi Zhou, James Xu, Amanda Prorok y Angela P. Schoellig (2021). «Learning to Fly — a Gym Environment with PyBullet Physics for Reinforcement Learning of Multi-agent Quadcopter Control». En: *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, págs. 7512-7519. DOI: 10.1109/IROS51168.2021.9635857. URL: <https://doi.org/10.1109/IROS51168.2021.9635857>.
- Pi, Chen-Huan, Yi-Wei Dai, Kai-Chun Hu y Stone Cheng (2021). «General Purpose Low-Level Reinforcement Learning Control for Multi-Axis Rotor Aerial Vehicles». En: *Sensors* 21.13, pág. 4560. DOI: 10.3390/s21134560. URL: <https://doi.org/10.3390/s21134560>.
- ResearchGate (2024). *Proximal Policy Optimization (PPO) architecture*. URL: https://www.researchgate.net/figure/Proximal-Policy-Optimization-PPO-architecture_fig2_385920419 (visitado 28-06-2026).
- Sánchez-López, José Luis, Martín Molina, Hriday Bavle, Carlos Sampedro, Ramón Suárez Fernández, David Palacios, Antonio Díaz-Moreno y Pascual Campoy (2017). «A Multi-Layered Component-Based Approach for the Development of Aerial Robotic Systems: The Aerostack Framework». En: *Journal of Intelligent & Robotic Systems* 88.2-4, págs. 683-709. DOI: 10.1007/s10846-017-0551-4. URL: <https://doi.org/10.1007/s10846-017-0551-4>.
- Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford y Oleg Klimov (2017). «Proximal Policy Optimization Algorithms». En: *arXiv preprint arXiv:1707.06347*. DOI: 10.48550/arXiv.1707.06347. URL: <https://arxiv.org/abs/1707.06347>.
- Song, Yunlong, Selim Naji, Elia Kaufmann, Antonio Loquercio y Davide Scaramuzza (2021). «Flightmare: A Flexible Quadrotor Simulator». En: *Proceedings of the 5th Conference on Robot Learning*. Vol. 155. Proceedings of Machine Learning Research. PMLR, págs. 1147-1157. URL: <https://proceedings.mlr.press/v155/song21a.html>.
- Song, Yunlong, Angel Romero, Matthias Müller, Vladlen Koltun y Davide Scaramuzza (2023). «Reaching the Limit in Autonomous Racing: Optimal Control versus Reinforcement Learning». En: *Science Robotics* 8.82, eadg1462. DOI: 10.1126/scirobotics.adg1462. URL: <https://doi.org/10.1126/scirobotics.adg1462>.

- Stable-Baselines3 Contributors (2026). *Stable-Baselines3 Documentation: PPO*. URL: <https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html> (visitado 25-06-2026).
- Sutton, Richard S. y Andrew G. Barto (2018). *Reinforcement Learning: An Introduction*. 2.^a ed. MIT Press. URL: <http://incompleteideas.net/book/the-book-2nd.html>.
- Towers, Mark, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan y Omar G. Younis (2024). «Gymnasium: A Standard Interface for Reinforcement Learning Environments». En: *arXiv preprint arXiv:2407.17032*. DOI: 10.48550/arXiv.2407.17032. URL: <https://arxiv.org/abs/2407.17032>.
- Yoo, Jaehyun, Dohyun Jang, H. Jin Kim y Karl H. Johansson (2021). «Hybrid Reinforcement Learning Control for a Micro Quadrotor Flight». En: *IEEE Control Systems Letters* 5.2, págs. 505-510. DOI: 10.1109/LCSYS.2020.3001663. URL: <https://doi.org/10.1109/LCSYS.2020.3001663>.

Apéndice A

Anexos

Los anexos recogen información complementaria que apoya la reproducibilidad de los experimentos sin interrumpir la exposición principal de la memoria. Se incluyen configuraciones representativas, comandos de ejecución y una matriz extendida de experimentos. Si se dispone de una captura de TensorBoard con métricas representativas, debe añadirse en este capítulo y referenciarse desde el Capítulo 6.

A.1. Configuraciones representativas

Las configuraciones Exp007, Exp008 y Exp008a representan la evolución desde un entrenamiento largo bloqueado por reset hacia ejecuciones de curriculum con mayor instrumentación. Las configuraciones Exp009–Exp012 corresponden a la fase posterior, en la que el reinicio del simulador ya está certificado y el foco se desplaza hacia la validez de la recompensa, la randomización de episodios, la reducción de salidas de límites y la continuidad de entrenamientos largos.

Config.	Objetivo	Duración	Rasgo principal
Exp007	Target desplazado estable.	20000 pasos.	Gate de reset tras eliminar fallback bloqueante.
Exp008	Target de 2 m.	20000 pasos.	Diagnóstico reforzado de reset y servicio del simulador.
Exp008a	Target de 0,5 m.	5000 pasos.	Curriculum inicial, yaw diagnóstico y <code>reset_max_vel</code> .
Exp009	Stack de reset certificado.	5000 pasos.	Primera ejecución PPO completa sobre el reinicio certificado.
Exp010	Reward mínima y randomización completa.	50000 pasos planeados.	Recompensa alineada con el tutor e hiperparámetros PPO revisados.
Exp011	Área segura y anti-saturación.	50000 pasos planeados.	Margen de muestreo y límite de distancia inicio-objetivo.
Exp012	Guards de límites y reanudación.	50000 pasos planeados.	Guard de acción, corredor vertical ampliado y supervisor con <i>resume</i> .

Cuadro A.1: Configuraciones representativas de la fase de estabilización.

A.2. Comandos representativos

Los comandos concretos dependen del estado local del entorno y del simulador, por lo que deben ejecutarse tras reiniciar de forma limpia la infraestructura AS2. Un flujo representativo consiste en lanzar el simulador, activar el entorno Python y ejecutar el script de entrenamiento o un validador acotado:

Listing A.1: Ejemplo de ejecución de entrenamiento PPO acotado.

```
1 python3 scripts/train_ppo.py \
2   --config configs/train_ppo_phase1_exp012.yaml \
3   --num-envs 1 \
4   --total-timesteps 1000
```

Para validar accionabilidad con una acción fija antes de un entrenamiento largo, se utiliza un smoke test acotado:

Listing A.2: Ejemplo de validación de accionabilidad con acción fija.

```
1 python3 scripts/validate_exp008_training_smoke.py \
2   --config configs/train_ppo_phase1_exp008a.yaml \
3   --episodes 3 \
4   --steps 12 \
5   --action 0.5 0.0 0.0 0.0
```

Estos comandos no deben interpretarse como recetas cerradas de entrenamiento, sino como ejemplos de ejecución reproducible. Antes de cada prueba con simulador conviene partir de un estado limpio de AS2 para evitar que estados residuales del controlador o de la dinámica contaminen el diagnóstico.

Para entrenamientos largos con posibilidad de caída del simulador o del proceso Python, la configuración más reciente se ejecuta mediante un supervisor que reinicia el stack de AS2, valida la disponibilidad del servicio de reinicio y reanuda desde el último *checkpoint* disponible:

Listing A.3: Ejemplo de entrenamiento supervisado con reanudación.

```
1 bash scripts/supervise_train.sh configs/train_ppo_phase1_exp012.yaml
```

A.3. Capturas de TensorBoard

TensorBoard resulta útil cuando la ejecución alcanza suficientes pasos para registrar escalares de Stable-Baselines3. En este trabajo no debe utilizarse como una figura decorativa, sino como apoyo para interpretar la evolución de cada experimento: recompensa media, longitud media de episodio, pérdidas de PPO y, si están disponibles, métricas derivadas del monitor. Las capturas deben relacionarse con la matriz de experimentos del Capítulo 6, porque una curva de recompensa o longitud de episodio solo es interpretable si se conocen las razones de terminación y los fallos de reset registrados en paralelo.

Para que la figura sea útil, se recomienda incorporar una captura o exportación de TensorBoard por experimento relevante. En particular, Exp010 y Exp011 permiten

mostrar por qué la alineación de recompensa y la corrección de la saturación de observaciones no bastaron para obtener aprendizaje estable. Exp012, cuando finalice, permitirá contrastar si la reducción de reinicios de servicio mejora la continuidad de los episodios y la tendencia de recompensa.

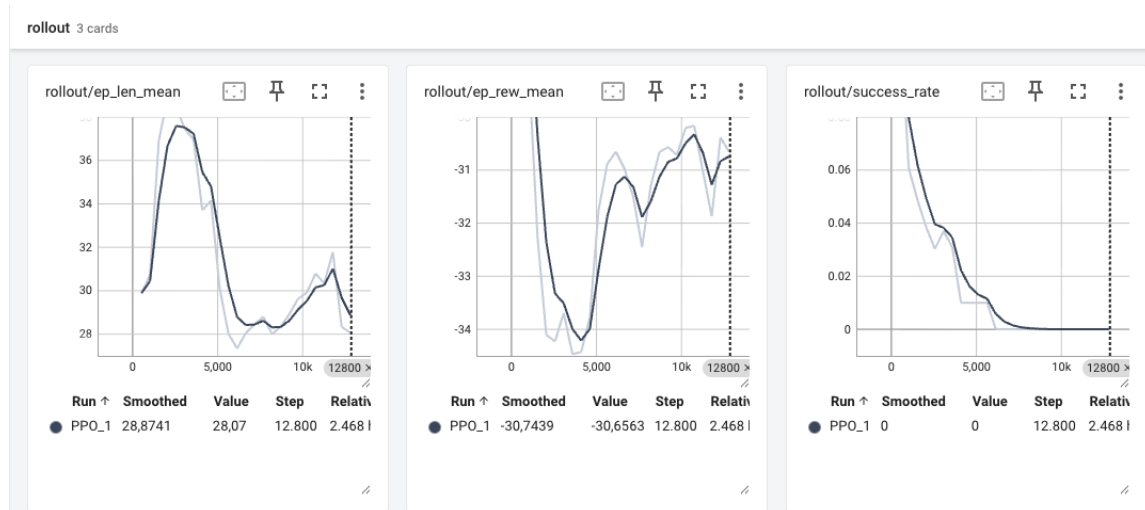


Figura A.1: Métricas de *rollout* de TensorBoard para Exp010. La tasa de éxito desciende hasta cero y la recompensa media no muestra una tendencia sostenida de mejora, por lo que la figura debe interpretarse como evidencia de diagnóstico y no como demostración de aprendizaje estable.



Figura A.2: Métricas de *rollout* de TensorBoard para Exp011. Aunque la configuración reduce la saturación inicial y mejora parcialmente la recompensa media frente a Exp010, la tasa de éxito vuelve a cero y las terminaciones por límites o baja altitud siguen dominando el entrenamiento.

A.3. Capturas de TensorBoard

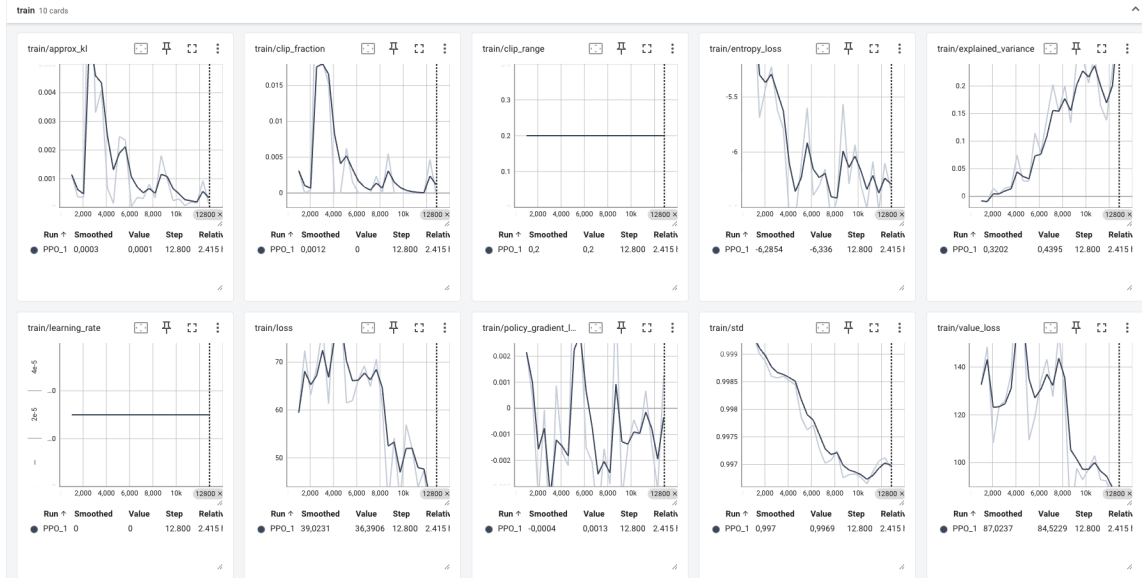


Figura A.3: Métricas internas de PPO para Exp010. Los indicadores de entrenamiento se mantienen en rangos razonables para una ejecución estable de PPO, pero esto no implica éxito de la tarea cuando las métricas de episodio muestran predominio de terminaciones no deseadas.

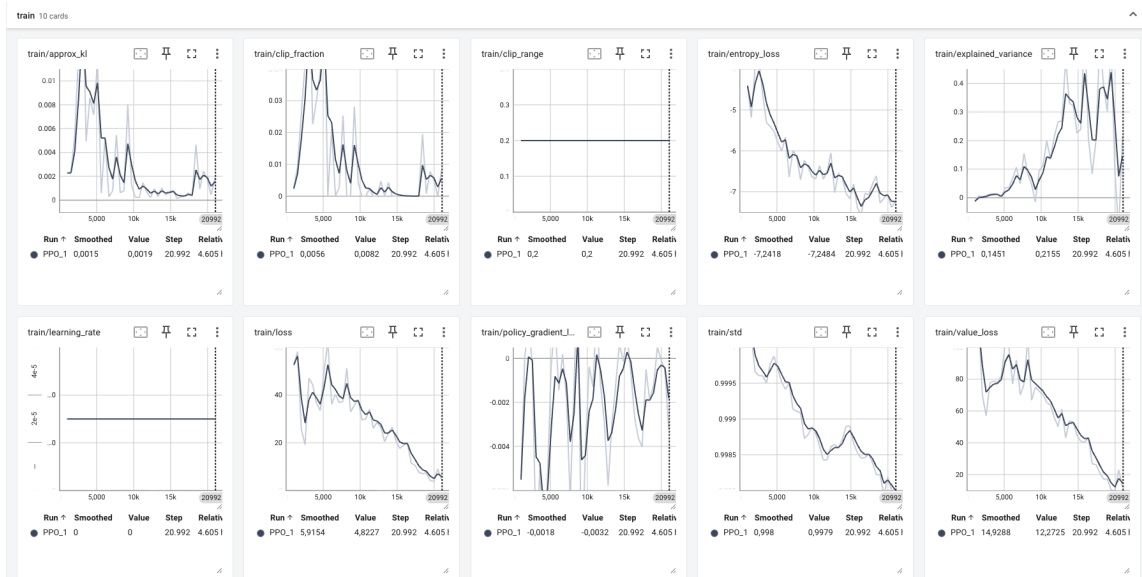


Figura A.4: Métricas internas de PPO para Exp011. La evolución de pérdidas, entropía y varianza explicada se utiliza como apoyo técnico, mientras que la interpretación principal del experimento se obtiene de las métricas de *rollout* y de los monitores de episodio.

A.4. Declaración de uso de herramientas de inteligencia artificial

Durante la realización del presente trabajo se han empleado herramientas de inteligencia artificial generativa como apoyo a tareas de programación, diagnóstico, organización de resultados y organización en la redacción. En particular, se ha utilizado OpenAI Codex en el entorno de desarrollo para asistencia sobre código, inspección del repositorio, generación de propuestas de modificación, revisión de errores y apoyo a la edición de la memoria [OpenAI, 2026b]. También se ha utilizado ChatGPT como asistente conversacional para estructurar explicaciones, revisar redacción y sintetizar criterios de presentación académica [OpenAI, 2026a].

El uso de estas herramientas no sustituye la autoría ni la responsabilidad académica del trabajo. Las decisiones de diseño, la interpretación de resultados, la selección de experimentos y la aceptación final de cambios se han revisado a partir del repositorio, de las trazas de ejecución, de los ficheros de configuración y de los logs experimentales. Las respuestas generadas por IA se han tratado como propuestas de trabajo que debían ser verificadas, corregidas o descartadas antes de incorporarse al documento o al código.

Herramienta	Tareas asistidas	Validación aplicada
OpenAI Codex	Inspección del repositorio, edición de LaTeX, revisión de errores de compilación y apoyo a cambios de código.	Revisión manual de los diffs, compilación de LaTeX y contraste con el repositorio.
ChatGPT	Estructuración de explicaciones, síntesis de resultados y apoyo a redacción académica.	Revisión del texto, eliminación de afirmaciones no respaldadas y contraste con evidencias experimentales.

Cuadro A.2: Resumen del uso de herramientas de inteligencia artificial generativa.

Entre los prompts más relevantes se encuentran los dirigidos a analizar el estado real del repositorio y del registro experimental, a identificar diferencias entre resultados de entrenamiento y resultados de infraestructura. Por otra parte para revisar la compilación LaTeX y para organizar la incorporación de figuras y tablas.

No se han utilizado las herramientas de inteligencia artificial como fuente bibliográfica primaria sobre aprendizaje por refuerzo, ROS 2, Aerostack2 o Stable-Baselines3. Las afirmaciones técnicas del cuerpo de la memoria se apoyan en referencias bibliográficas, documentación técnica, código implementado y evidencia experimental. Cuando se ha incorporado texto redactado con apoyo de IA, este se ha revisado y adaptado para mantener el tono académico, la precisión terminológica y la coherencia con los resultados disponibles.